

第 12 章: if and match Selections (if 与 match 选择)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 本章是全书第一次系统学习复合语句 (compound statement) 的章节——介绍 Python 用于"根据测试结果选择动作"的两条语句: if/elif/else (任意的选择逻辑) 和 match/case (Python 3.10 新增的结构模式匹配)。它同时补全了第 9 章遗留的真值测试细节、深入讲解了第 10 章引入的语句语法模型 (缩进、续行), 并介绍了 if/else 三元表达式。学完本章, 你就掌握了 Python 程序"逻辑骨架"的完整拼图。

12.1 开篇引言 (Chapter Introduction)

英文原文

This chapter presents Python's two statements used for selecting from alternative actions based on test results:- if / elif / else — The main selection workhorse in most programs, capable of coding arbitrary logic-match / case — A tool for narrower multiple-choice selection, with advanced matching operations Because

this is our first in-depth look at compound statements—statements that embed other statements—we will also explore the general concepts behind the Python statement syntax model here in more detail than we did in the introduction in Chapter 10. Because the if statement introduces the notion of tests, this chapter will also deal with Boolean expressions, cover the "ternary" if expression, and fill in some details on truth tests in general.

中文翻译

本章介绍 Python 用于"根据测试结果从备选动作中选择"的两条语句：- if / elif / else ——大多数程序中的主力选择语句，能够表达任意的逻辑- match / case ——面向更狭窄的多选场景的工具，带有高级匹配操作因为这是我们对复合语句（compound statement）——即嵌入了其他语句的语句——的第一次深入接触，我们也会在这里比第 10 章的引言更详细地探讨 Python 语句语法模型背后的通用概念。由于 if 语句引入了"测试"（test）的概念，本章还将涉及布尔表达式（Boolean expression）、介绍"三元"if 表达式，并补充一般真值测试的细节。

深度理解

·核心概念：本章回答程序最基础的问题——"根据条件决定做什么"。if 负责任意逻辑分支；match 负责"按值多选"。两者构成了 Python 全部控制流（control flow）的选择部分。

- 底层视角："复合语句"是本章的关键词：语句里嵌套语句。Python 靠"冒号 + 缩进"识别嵌套结构，而不是像 C/Java 那样靠 {}。这意味着 Python 解释器在词法分析阶段就要统计每行的缩进深度，这是它与其他语言编译流程最显眼的差异。
- 设计原因：作者把"真值测试、布尔表达式"放在 if 的章节里讲，是因为这些概念只有被 if 真正用起来才有意义——先有需求，再讲工具。
- 实际问题：所有程序都离不开选择逻辑。本章给出三条"多选"路线：if/elif 链（逻辑最自由）、字典索引（数据驱动、可动态构建）、match（语法显式但功能受限）。
- 初学者误区：以为 match 是 if 的替代品。实际上 match 只能做"按值匹配"，做不了 if 那种"任意表达式组合"的逻辑——它们是互补工具，不是上下位关系。

12.2 if 语句 (if Statements)

英文原文

In simple terms, the Python if statement selects actions to perform. Along with its if expression counterpart, it's the primary selection tool in Python and represents much of the logic a Python program possesses. It's also our first compound statement. Like all such statements, the if statement may contain other statements, including other ifs. In fact, Python lets you combine statements in a program both sequentially (so t

that they execute one after another), and in an arbitrarily nested fashion (so that they execute only under certain conditions, such as selections and loops).

中文翻译

简单来说, Python 的 if 语句选择要执行的动作。连同它的 if 表达式 (if expression) 变体一起, 它是 Python 中最主要的选择工具, 代表了一个 Python 程序所拥有的大部分"逻辑"。它也是我们遇到的第一个复合语句。与所有这类语句一样, if 语句可以包含其他语句, 包括其他的 if。事实上, Python 允许你在程序中组合语句: 既可以"顺序"组合 (让它们一条接一条执行), 也可以任意"嵌套"组合 (让它们只在特定条件下执行, 比如选择和循环)。

深度理解

- 核心概念: if 是程序的"岔路口", 程序的所有"决策"都由它 (和循环条件) 承担。
- 底层实现: CPython 把 if 编译成字节码: 先计算测试表达式的值, 然后执行条件跳转指令 (如 POPJUMPIFFALSE), 把执行流送到某个分支代码段。每个分支块编译成一段独立的字节码, 块与块之间靠跳转串联。
- 设计原因: "顺序 + 嵌套"的组合方式与 CPU 的指令跳转模型天然对应, 简单而高效; 同时让代码能表达任意的逻辑深度。
- 实际问题: 输入校验、权限判断、错误处理、业务分支...
...凡是"根据情况做不同的事", 都是 if 的用武之地。

·初学者误区：以为 if 的测试必须是布尔值。其实任何对象都可以被直接测试真伪（1、'abc'、[1,2] 都是真），这是 Python 与其他语言非常不同的习惯。

12.2.1 通用格式 (General Format)

英文原文

The Python if is typical of if statements in most procedural languages. It takes the form of an if test, followed by one or more optional elif (for "else if") tests, and a final and optional else block. The tests and the else part each have an associated block of nested statements, indented under a header line. When the if statement runs, Python executes the block of code associated with the first test that evaluates to true, or the else block if all tests prove false. The general form of an if statement looks like this:

```
if test1:                # Main if test
    statements1          # Associated block

elif test2:              # Optional elif test(s)
    statements2          # Associated block

else:                    # Optional else default
    statements3          # Associated block
```

中文翻译

Python 的 if 与大多数过程式语言中的 if 语句相同。它的形式是：一个 if 测试，后面跟一个或多个可选的 elif（即 "else if"）测试，最后跟一个可选的 else 块。每个测试和 el

se 部分都各自关联一个嵌套语句块，缩进在头部行 (header line) 之下。当 if 语句运行时，Python 执行与"第一个求值为真"的测试相关联的代码块；如果所有测试都为假，则执行 else 块。if 语句的通用形式如下：

```
if test1:           # if
    statements1     #

elif test2:         # elif
    statements2     #

else:               # else
    statements3     #
```

深度理解

·核心概念：if/elif/else 是一条"自上而下、首真即停"的测试链：从第一个测试开始依次求值，命中第一个为真的就执行其块，其余全部跳过。

·底层实现：编译时，elif 会被展开成"跳转"结构——test 1 为假就跳到 test2 的代码段，test2 为假再跳到 else 段。这与 C 语言中 else if 链的编译结果在逻辑上完全等价。

·设计原因："首真即停"保证了各分支互斥 (mutually exclusive)，也避免了对后续条件做无谓求值。

·实际问题：elif 本质是"else 后面再跟一个 if"的语法糖，它把多分支代码从"无限向右缩进"的箭头形 (arrow-shaped) 代码中解放出来，保持了可读性。

·初学者误区：1) 以为 elif 和 else 都必须写——其实除了初始 if 测试和它的语句块，其余部分全是可选的；2) 忘

了 elif 链一旦命中，后面的分支就再也不会被执行。

12.2.2 基础示例 (Basic Examples)

英文原文

To demonstrate, let's turn to a few simple examples of the if statement at work, in the REPL as usual. All parts are optional, except the initial if test and its associated statements. Thus, in the simplest case, the other parts are omitted:

```
$ python3
> >> if 1:

...     print('true')
...
true
```

As you've seen before, the prompt changes to ... for continuation lines in many REPLs; in IDLE, you'll simply drop down to an indented line instead (and tap Back space to back up when needed). A blank line, input by pressing Enter twice, terminates and runs the entire statement in most interactive interfaces. Remember that 1 is Boolean true (as we'll review later, the word True is its equivalent), so this statement's test always succeeds. To handle a false result here, code the else to be run when the if test is not true:

```
> >> if not 1:

...     print('true')
... else:
...     print('false')
...
false
```

If you're working along: this chapter would like to omit all the ... prompts for easier media copy and paste, but this would throw off indentation of associated lines like else. Instead, prompts are left off where possible, and code of some longer examples is listed without prompts, before its output. The book's example package also has code sans prompts. Here's a more complex if statement with all its optional parts present; it aims to display the roots of today's mobile operating systems (though it's not fully inclusive, and is prone to grow dated in a discriminating future near you):

```
if os in ['iOS', 'iPhoneOS']:
    print('macOS')
elif mode == 'mobile' and os != 'Windows':
    print('Linux')
else:
    print('unknown?')
> >> os, mode = 'Windows', 'mobile'>>...insert the code above here...

unknown?
```

This multiline statement extends from the if line through the block nested under the else. When it's run, Python executes the statements nested under the first

test that is true, or the else part if all tests are false (in this example, they are). In practice, both the elif and else parts may be omitted, and there may be more than one statement nested in each section. Note that the words if, elif, and else are associated by the fact that they line up vertically, with the same indentation (ignoring the REPL prompts you may see if you paste and run this live). The and expression in the preceding example is true if the expressions on its left and right sides are both true (more on such logical tests ahead). The if statement can also be nested to code choices that depend on others, and arbitrary logic in general. The following, for example, first ensures that it's dealing with a mobile before checking specific system names—logically speaking, there is an implied and between the enclosing and nested ifs:

```
if mode == 'mobile':
    if os == 'Android':
        print('Linux')
    elif os != 'Windows':
        print('macOS')
> >> os, mode = 'Android', 'mobile'>>...insert the code above here...
```

Linux

中文翻译

为了演示，让我们照例在 REPL 中看几个简单的 if 语句实例。除了初始的 if 测试及其关联语句，所有部分都是可选的。因此，在最简单的情况下，其余部分可以省略：

```
$ python3
>>> if 1:
...     print('true')
...
true
```

正如你之前所见，在许多 REPL 中，续行提示符会变成 ...；在 IDLE 中，你只会落到一个缩进行上（需要时按 Backspace 回退）。在大多数交互式界面中，输入一个空行（按两次 Enter）就能结束并运行整条语句。记住 1 是布尔真值（后面我们会复习，单词 True 是它的等价物），所以这条语句的测试永远成功。要处理结果为假的情况，可以编写 else，让它在 if 测试不为真时执行：

```
>>> if not 1:
...     print('true')
... else:
...     print('false')
...
false
```

如果你正在跟着做：本章本想省略所有 ... 提示符以便复制粘贴，但这会打乱 else 等关联行的缩进。因此，尽量省略提示符，一些较长示例的代码在输出之前不带提示符列出。本书的示例包中也有无提示符的代码。下面是一个所有可选部分都齐全的、更复杂的 if 语句；它旨在展示当今移动操作系统的“血统”（虽然并不完全周全，而且在某个讲究的未来里容易过时）：

```

if os in ['iOS', 'iPhoneOS']:
    print('macOS')
elif mode == 'mobile' and os != 'Windows':
    print('Linux')
else:
    print('unknown?')

>>> os, mode = 'Windows', 'mobile'
>>> .....
unknown?

```

这条多行语句从 if 行一直延伸到嵌套在 else 之下的语句块。运行时，Python 执行"第一个为真"的测试下嵌套的语句；如果所有测试都为假，则执行 else 部分（本例正是如此）。实践中，elif 和 else 部分都可以省略，每个部分下面也可以嵌套多条语句。注意 if、elif、else 这几个词是靠"垂直对齐、缩进相同"关联在一起的（忽略你粘贴运行时看到的 REPL 提示符）。前面例子中的 and 表达式在左右两侧表达式都为真时为真（关于这类逻辑测试后面还有更多）。if 语句也可以嵌套，用来编写"依赖其他选择的选择"，以及一般意义上的任意逻辑。例如下面的代码先确保处理的是移动设备，再检查具体的系统名称——从逻辑上讲，外层 if 和内层 if 之间隐含了一个 and：

```

if mode == 'mobile':
    if os == 'Android':
        print('Linux')
    elif os != 'Windows':
        print('macOS')

>>> os, mode = 'Android', 'mobile'
>>> .....
Linux

```

深度理解

- 核心概念：基础示例展示了三件事：任何对象可直接当测试（if 1:）、可选部分的省略、if 的嵌套（外层与内层之间隐含 and 关系）。
- 底层实现：if 1: 编译时，测试表达式 1 先被压栈，再执行 POPJUMPIFFALSE——但 1 永远为真，跳转永不发生，print('true') 必然执行。嵌套的 if 在字节码层面就是内层块的外层条件不满足时整段跳过。
- 设计原因：Python 允许直接测试对象（if X:）而不是写 if X != "" 或 if X != None，因为"非零即真、非空即真"是 Python 统一的真值哲学，简化了 90% 的判空代码。
- 实际问题：嵌套 if 适合"层层过滤"的逻辑；但嵌套过深会降低可读性，通常三层以上就该考虑用函数拆解或 and 合并条件。
- 初学者误区：1) 在 REPL 里输入 if 后忘记敲两次回车导致语句不执行；2) 误以为 elif 可以脱离 if 单独使用；3) 粘贴带 ... 提示符的代码导致缩进错乱。

代码分析

```
if 1:                                # 1
    print('true')                    #

if not 1:                             # not 1
    print('true')                    #
else:
    print('false')                   #

os, mode = 'Windows', 'mobile'
```

```
if os in ['iOS', 'iPhoneOS']:          # 'Windows'
    print('macOS')
elif mode == 'mobile' and os != 'Windows':  # True and False ->
    print('Linux')
else:
    print('unknown?')    # else unknown?
```

解释器如何处理：CPython 对 if/elif/else 采用"顺序求值 + 条件跳转"的编译策略。每个测试表达式求值后立即压栈，紧接着生成 POPJUMPIFFALSE 指令——若结果为假，PC（程序计数器）直接跳到下一个分支的代码段开头，跳过当前块的所有语句。and 表达式本身也编译为短路跳转：左操作数为假时，右操作数根本不会被求值（字节码中不存在对应的指令路径）。这种设计让 Python 的 if 链在运行时只做最少量的工作，性能开销极低。

设计思想：第一个测试决定"走哪条路"，测试之间靠跳转而非"逐条执行再丢弃"来连接，这正是图灵机模型里"选择"的标准实现，也是所有过程式语言共通的底层逻辑。

12.3 多选逻辑 (Multiple-Choice Selections)

英文原文

Until Python's version 3.10, it had no multiple-choice selection statement similar to a "switch" in some other languages that selects an action based on a variable's value. As you'll learn ahead, Python today has sprouted a match statement that achieves the same goal

Is (and substantially more!). Even so, multiple-choice logic can usually be coded just as easily by a series of if/elif tests and occasionally by indexing dictionaries or searching lists. Because dictionaries and lists can be built at runtime dynamically, they are often more flexible than hardcoded if (or match) logic in your script. The following, for instance, picks an operating system's release year, more or less, from its name:

```
>>> choice = 'Windows'
>>> print({'macos': 2001,          # A dictionary-based 'switch'
...       'Linux': 1991,          # Use get() for a default (ahead)
...       'Windows': 1985}[choice])
1985
```

Although it may take a few moments for this to sink in the first time you see it, this dictionary is a multiple-choice branch—indexing on the key choice branches to one of a set of values, much like a "switch" statement in other languages. An almost equivalent but more verbose Python if statement might look like the following:

```
if choice == 'macos':          # The equivalent if statement
    print(2001)
elif choice == 'Linux':
    print(1991)
elif choice == 'Windows':
    print(1985)
else:
    print('Bad choice')

>>> ...insert the code above here...
1985
```

Though it's perhaps more readable, the potential downside of an if like this is that, short of constructing it as a string and running it with tools like the eval or exec tools noted in Chapter 10, it cannot handle choices unknown until the program runs as easily as a dictionary. In more dynamic programs, data structures offer added flexibility.

中文翻译

直到 Python 3.10 之前，Python 都没有类似其他语言中 "switch" 那样的、根据变量值选择动作的多选语句。你稍后会学到，如今的 Python 长出了 match 语句，实现了同样的目标（而且远不止于此！）。即便如此，多选逻辑通常也可以直接用一系列 if/elif 测试来实现，偶尔还可以用字典索引或列表搜索来实现。因为字典和列表可以在运行时动态构建，它们通常比脚本中硬编码的 if（或 match）逻辑更灵活。例如，下面这段代码按操作系统名称找出（大致）对应的发布年份：

```
>>> choice = 'Windows'
>>> print({'macos': 2001,          # 'switch'
...       'Linux': 1991,          # get()
...       'Windows': 1985}[choice])
1985
```

第一次看到时可能要多想一会儿才能明白：这个字典就是一个多选分支——用键 choice 索引，就分支到一组值中的某一个，很像其他语言中的 "switch" 语句。一个几乎等价、但更啰嗦的 Python if 语句可能长这样：

```
if choice == 'macos':           # if
    print(2001)
elif choice == 'Linux':
    print(1991)
elif choice == 'Windows':
    print(1985)
else:
    print('Bad choice')

>>> .....
1985
```

虽然它可能更易读，但这种 if 的潜在缺点是：除非把代码构造成字符串、再用第 10 章提到的 eval 或 exec 工具来运行，否则它无法像字典那样轻松地处理“程序运行时才已知”的选择。在更动态的程序中，数据结构提供了额外的灵活性。

深度理解

- 核心概念：这是 Python 版的 "switch 替代方案"——三种实现：if/elif 链、字典索引、match。字典索引的本质是数据驱动分支：键 → 值，一次哈希查找即完成选择。
- 底层实现：字典索引 d[choice] 是对 choice 做哈希 (hash)，再在哈希表槽位中定位——时间复杂度 $O(1)$ 。if/elif 链则是顺序比较，平均 $O(n)$ 。分支数多时，字典在性能上更有优势。
- 设计原因：Python 刻意多年不引入 switch，是因为字典 + if 已经覆盖了绝大多数场景，且字典是数据（可运行时构建、可扩展），if 是代码（编译时固定）。数据结构比代码更灵活。

·实际问题：当选择项是"程序运行时从配置/网络/用户输入才知道的"时，字典方案天然适配；if 方案则要写死所有分支。

·初学者误区：1) 以为字典做 switch 是"黑魔法"——其实只是普通的键索引；2) 不知道字典索引遇到不存在的键会抛 `KeyError`，需要配合 `get()` 或 `try`。

代码分析

```
choice = 'Windows'
print({'macos': 2001,          # 'switch'
      'Linux': 1991,          # get()
      'Windows': 1985}[choice]) # choice
# 1985

if choice == 'macos':        # if
    print(2001)
elif choice == 'Linux':
    print(1991)
elif choice == 'Windows':
    print(1985)
else:
    print('Bad choice')      #
```

解释器如何处理：字典索引的字节码是 `BINARYSUBSCR`：解释器先对 `choice` 的值（字符串'Windows'）计算哈希码，然后在字典的哈希表里定位槽位，直接取出 1985——一次查找即完成分支，不需要比较链。而 if/elif 链则要依次执行三次 `==` 比较（每比较一次字符串都有长度扫描开销）。两者结果相同，但查找路径完全不同：一个是"哈希定位"，一个是"线性扫描"。

设计思想：把"选择逻辑"从"代码"搬进"数据"，是程序员从入门走向进阶的标志性思维转变——数据可以来自文件、网络、用户输入，代码不能。这也是后面讲"跳转表" (jump table, 字典存函数) 的思想雏形。

12.3.1 处理 switch 的默认情况 (Handling switch defaults)

英文原文

Notice the else clause on the if here to handle the default case when no key matches. As demoed in Chapter 8, dictionary defaults can be coded with in expressions, get method calls, or exception catching with the try statement introduced in the preceding chapter. All of these same techniques can be used here to code a default action in a dictionary-based multiple choice. As a review in the context of this role, here's the get scheme at work with defaults (which also uses the more compact dict call to make the same dictionary):

```
> >> branch = dict(macos=2001, linux=1991, windows=1985) >> pri...
```

```
1985
```

```
> >> print(branch.get('Solaris', 'Bad choice'))
```

```
Bad choice
```

An in membership test in an if statement can have the same default effect:

```
> >> choice = 'AmigaOS'>> if choice in branch:

...     print(branch[choice])
... else:
...     print('Bad choice')
...
Bad choice
```

And the try statement is a general way to handle dictionary-based defaults by catching and handling the errors they'd otherwise trigger (for more on exceptions, see Chapter 11's overview and Part VII's full treatment):

```
> >> choice = 'GEM'>> try:

...     print(branch[choice])
... except KeyError:
...     print('Bad choice')
...
Bad choice
```

中文翻译

注意这里的 if 上用来处理"没有键匹配"默认情况的 else 子句。正如第 8 章演示的，字典默认值可以用 in 表达式、get 方法调用、或上一章引入的 try 语句捕获异常来实现。所有这些技术都可以用在这里，为基于字典的多选编写默认动作。作为这种角色的复习，下面是用 get 配合默认值的方案（它还用了更紧凑的 dict 调用来创建同样的字典）：

```
>>> branch = dict(macos=2001, Linux=1991, Windows=1985)
>>> print(branch.get('Windows', 'Bad choice'))
1985
>>> print(branch.get('Solaris', 'Bad choice'))
Bad choice
```

if 语句中的 in 成员测试也能产生同样的默认效果：

```
>>> choice = 'AmigaOS'
>>> if choice in branch:
...     print(branch[choice])
... else:
...     print('Bad choice')
...
Bad choice
```

而 try 语句是通过捕获并处理本会触发的错误来给字典默认值兜底的通用方法（关于异常，详见第 11 章概览和第七部分的完整讲解）：

```
>>> choice = 'GEM'
>>> try:
...     print(branch[choice])
... except KeyError:
...     print('Bad choice')
...
Bad choice
```

深度理解

·核心概念：字典索引的“默认值”处理有三套惯用法：get(key, default)（最简洁）、if key in dict: 守卫（可读性最好）、try/except KeyError（最通用，能顺带处理其他错误）。

·底层实现：dict.get(key, default) 在 CPython 内部同样是哈希查找，只是找不到时返回默认参数而不是抛 KeyErr

or; try/except 则依赖解释器建立的异常处理帧 (exception frame) , 在 C 层面设置跳转目标, 捕获异常时执行 except 块。

·设计原因: 把"默认行为"做成三种可选方案, 体现了 Python "多种方式、一种最佳"的哲学——场景不同, 选择不同: 只看结果用 get, 要区分"键在不在"用 in, 要连错误一起兜底用 try。

·实际问题: 真实代码中"查表 + 兜底"是最常见的模式 (配置解析、状态机、路由表) , 掌握这三种写法可以应对绝大多数需求。

·初学者误区: 1) 直接用 branch[choice] 而不考虑键不存在会崩溃; 2) 不知道 dict(...) 关键字参数形式要求键必须是合法标识符 ('macos' 可以, '123' 不行) 。

12.3.2 处理更大的动作 (Handling larger actions)

英文原文

As you can tell, dictionaries are good for associating values with keys, but what about the more complicated actions you can code in the statement blocks associated with if statements? In Part IV, you'll learn that dictionaries can also contain functions to represent more complex actions and implement general "jump tables." Such functions appear as dictionary values, may be coded as function names or inline lambdas, and are run by simply adding parentheses to trigger th

air actions. Here's an abstract sample of this technique, but stay tuned for a rehash in Chapter 19 after you've learned more about function definition:

```
def action(): ...
def default(): ...
branch = {'Android': lambda: ...,          # A table of callable f...
          'iOS': action,                  # Via inline lambdas, o...
          'Symbian OS': lambda: ...}
choice = 'Android'
branch.get(choice, default)()             # Fetch and run associa...
```

Although dictionary-based multiple-choice branching is useful in programs that deal with more dynamic data, most programmers will probably find that coding an if statement is a straightforward way to perform this task. As a rule of thumb in coding, when in doubt, err on the side of simplicity and readability; it's the "Pythonic" way. And the punch line here, of course, is that the match statement may handle basic multiple-choice selections better today—though it's doesn't do general logic like if, can't handle dynamic data like dictionary indexing, and comes with substantial extra convolution for other roles. You'll have to move on to the next section to see all this for yourself.

中文翻译

如你所见，字典擅长把值与键关联起来，但 if 语句块里那些更复杂的动作该怎么办呢？在第四部分你会学到：字典也可以包含函数（function）来表示更复杂的动作，从而实现通用的“跳转表”（jump table）。这类函数作为字典的值出

现，可以用函数名或内联 lambda 编写，运行时只需加上括号就能触发它们的动作。这是该技术的一个抽象示例，在第 19 章你学到更多函数定义知识之后，还会有更详细的复述：

```
def action(): ...
def default(): ...
branch = {'Android': lambda: ...,           #
          'iOS': action,                    # lambda def ...
          'Symbian OS': lambda: ...}
choice = 'Android'
branch.get(choice, default)()              #
```

虽然基于字典的多选分支在处理更动态数据的程序中很有用，但大多数程序员大概会发现，编写 if 语句是完成这个任务的直白方式。作为编程的一条经验法则：拿不准的时候，宁可选择简单和可读；这才是 "Pythonic" 的方式。当然，这里的点睛之笔是：match 语句今天可能更好地处理基础的多选场景——虽然它不能像 if 那样做任意逻辑，不能像字典索引那样处理动态数据，而且为了其他角色背负了相当大的额外复杂度。你得翻到下一节才能亲自看到这一切。

深度理解

·核心概念："跳转表"是字典多选的进阶形态：值存函数，查表之后加括号调用。这是"以数据代替代码"哲学的极致——一个表就顶一整棵 if-elif 树。

·底层实现：函数是第一类对象 (first-class object)，可以像整数、字符串一样存进字典。branch.get(choice, default) 返回函数对象，再 () 调用它；若键不存在，get 返回 default 函数并被调用。

·设计原因：跳转表把"分支逻辑"完全数据化：新增一个选择项 = 往表里加一项，不用改动任何控制流代码——这在插件架构、命令处理器里非常常见。

·实际问题：命令行的子命令分发（如 git add/commit）、HTTP 路由表、状态机迁移表，都是跳转表的实际应用。

·初学者误区：1) 忘了"取函数"和"调函数"是两步——branch['x'] 拿到的是函数对象，branch['x]() 才是执行；2) 以为 lambda 可以做任意复杂的事——它只能放表达式。

代码分析

```
def action(): ...           # def
def default(): ...
branch = {'Android': lambda: ...,      #
         'iOS': action,                # lambda def ...
         'Symbian OS': lambda: ...}
choice = 'Android'
branch.get(choice, default)()          #
```

解释器如何处理：def action() 在编译时生成一个函数对象并绑定到名字 action；lambda: ... 同样是函数对象，只是没有名字（匿名）。构建字典时，解释器把函数对象的引用作为值存入哈希表。branch.get(choice, default) 做哈希查找，返回键'Android' 对应的 lambda 函数对象；随后的 () 是调用操作符——解释器压入新的函数调用栈帧（frame），执行 lambda 体，返回后弹出帧。整个过程就是"查表 → 取函数 → 建帧 → 执行 → 销毁帧"。

设计思想："函数作为值"是函数式编程的核心能力。作者在此埋下伏笔，第 19 章（函数）会系统展开——这里先

让你在 if 的语境里看到"数据驱动的分支"能走多远。

12.4 match 语句 (match Statements)

英文原文

For most of Python's three-decade career, it resisted adding a multiple-choice selection statement, in large part because of all the options for coding such logic with existing tools that we just explored. Those options permeate vast amounts of Python code and remain relatively simple techniques that are perfectly valid to use in Python code written today. Nevertheless, programming languages have a tendency to get caught up in arms races with each other—copying other languages' features and eroding their own distinctions in the process, and often for no better reason than familiarity with other tools. One of the fruits of this process is the match statement, new as of Python 3.10. At its basic level, match is a potentially useful tool that adds a multiple-choice statement to Python. At this level, it works very much like "switch" statements in other languages and can be used instead of both if/elif/else combos and dictionary indexing in some contexts. In its full-blown form, however, match implements something known as structural pattern matching, which quickly falls off a complexity cliff, and seems a highly convoluted answer to a question that most Pyt

hon programmers never asked. Especially for newcomers, it's a lot to justify. Because of all that, this section is going to focus on the basic roles of match, and touch on its advanced pattern-matching roles only briefly, with delegation to Python's manuals for more of the story.

中文翻译

在 Python 三十年的职业生涯中，大部分时间里它都抵制添加多选语句，很大程度上是因为我们刚才探索过的、用现有工具编写这类逻辑的各种方案。这些方案渗透在大量的 Python 代码中，至今仍是完全有效的、相对简单的技术。然而，编程语言之间有一种陷入“军备竞赛”（arms race）的倾向——互相抄袭其他语言的特性，并在此过程中侵蚀自己的特色，而且往往仅仅因为对别的工具更熟悉。这一过程的产物之一就是 match 语句，它是 Python 3.10 的新特性。在基本层面上，match 是一个潜在有用的工具，为 Python 增加了一条多选语句。在这个层面上，它很像其他语言中的“switch”语句，在某些场景下可以替代 if/elif/else 组合和字典索引。然而在完全形态下，match 实现了所谓的结构模式匹配（structural pattern matching），它会迅速跌下复杂度的悬崖，看起来像是对“大多数 Python 程序员从未问过的问题”给出的一个高度绕弯的答案。尤其是对新手而言，很难论证它的价值。正因为如此，本节将聚焦 match 的基础角色，仅简要提及它的高级模式匹配角色，更完整的介绍则委托给 Python 官方手册。

深度理解

·核心概念：match 是 Python 3.10（2021 年）新增的多选语句。作者的态度很直白：基本用法有用，高级的结构模式匹配"复杂得离谱"。

·底层实现：match 语句会被编译为一连串的"模式匹配测试"字节码序列：逐个 case 求值、比较，命中即跳转执行该 case 块，否则落入下一个 case。每个 case 头部的模式在编译期被分析为具体的匹配操作（比较、拆包、类型检查等）。

·设计原因：Python 三十年没有 switch，是因为字典和 if 已经够用；引入 match 更多是"军备竞赛"压力下的产物——其他语言（Rust、Scala、C#、Java 17 等）都有模式匹配，社区呼声推动 PEP 634-636 落地。

·实际问题：基本用法解决"按值多选"的显式语法需求；高级用法解决"按结构提取数据"的问题（如解析 JSON、AST 节点处理）。

·初学者误区：1) 以为 match 一定要用，其实简单多选用 if 或字典往往更清晰；2) 以为 case 是普通变量名——它是通配符；3) 忽略版本要求——match 需要 Python 3.10+，老代码库里会直接 SyntaxError。

12.4.1 基础 match 用法 (Basic match Usage)

英文原文

The good news is that match's basic usage is straight forward. In its first-level form, match works the same

as both an if statement and a dictionary index—like the following abstract snippets that emulate traffic lights in some locales (to run most examples here live, first assign variables to one of the options listed in their opening comments):

```
# state = 'go' or 'stop' or other
if state == 'go':
    print('green')
elif state == 'stop':                # if-based multiple choice
    print('red')
else:
    print('yellow')

colors = dict(go='green', stop='red')    # Dictionary-based multip...
print(colors.get(state, 'yellow'))
```

The equivalent match statement provides explicit syntax for such multiple-choice logic, at the cost of extra lines and extra indentation:

```
match state:
    case 'go':
        print('green')                # match-based multiple choice: ...
    case 'stop':
        print('red')
    case _:
        print('yellow')
```

This statement works like this: match first evaluates the expression given in its header line (e.g., state) and then compares its result to values given in case header lines indented below it (e.g., 'go')—one after another, and top to bottom. As soon as a first match is found, the block of code nested under the matching cas

e is run, and the entire match is exited. If no case values match, the match statement either runs the block under the case with value (which provides a fallback default, and must appear last), or simply exits silently if no case is present. As usual, case blocks can contain multiple statements and nesting, and match itself can be nested in other statements. Moreover, case headers can also designate multiple values separated by `|` ("or") which are all checked for a match, name a variable to be assigned to the matched value with `as`, and give a simple variable that is assigned the match expression's result and always matches (and hence ends the statement, much like the anonymous `_`):

```
# state = 'go' or 'proceed' or 'start', or 'stop' or 'halt', or...
match state:
    case 'go' | 'proceed' | 'start':          # Match any one of these...
        print('green')                      # First left-to-right match
        print('means go')
    case 'stop' | 'halt' as what:            # Match any, and assign...
        print('red')                        # what outlives match i...
        print('means', what)
    case other:                             # Set other to state, a...
        print('catchall', other)           # other outlives match ...
```

In general, variables (like this example's `what` and `other`) embedded in case headers and assigned during a successful match outlive the match statement itself: they can be used in code after the match exits, as long as that code is part of the same scope—which roughly means the same module or function, per coverage later in this book.

中文翻译

好消息是 `match` 的基础用法很直接。在它的一级形态下，`match` 的工作方式与 `if` 语句和字典索引相同——就像下面这些模拟某些地区交通灯的抽象代码片段（要在线运行这里的大多数示例，先把变量赋成它们开头注释里列出的选项之一）：

```
# state = 'go' 'stop'
if state == 'go':
    print('green')
elif state == 'stop':           # if
    print('red')
else:
    print('yellow')

colors = dict(go='green', stop='red')    #
print(colors.get(state, 'yellow'))
```

等价的 `match` 语句为这种多选逻辑提供了显式语法，代价是更多的行数和更多的缩进：

```
match state:
    case 'go':
        print('green')           # match 3.10+
    case 'stop':
        print('red')
    case _:
        print('yellow')
```

这条语句的工作方式如下：`match` 首先计算其头部行中给出的表达式（如 `state`），然后把结果与下方缩进的 `case` 头部行中给出的值（如 `'go'`）——比较——从上到下逐个进行。一旦找到第一个匹配，就运行该 `case` 下嵌套的代码块，并退出整个 `match`。如果没有 `case` 值匹配，`match` 语

句要么运行值为的 case 下的块（提供兜底默认，必须放在最后），要么在没有 case 时静默退出。与往常一样，case 块可以包含多条语句和嵌套结构，match 本身也可以嵌套在其他语句中。此外，case 头部还可以指定用 | ("或") 分隔的多个值——它们都会被检查是否匹配；可以用 as 给匹配到的值命名一个变量；还可以写一个简单变量，它会被赋值为 match 表达式的结果并永远匹配（因此会结束这条语句，很像匿名）：

```
# state = 'go' 'proceed' 'start' 'stop' 'halt'
match state:
    case 'go' | 'proceed' | 'start':          #
        print('green')                       #
        print('means go')
    case 'stop' | 'halt' as what:             # what
        print('red')                         # what match ...
        print('means', what)
    case other:                               # other state
        print('catchall', other)             # other match ...
```

一般来说，嵌入在 case 头部、在成功匹配时被赋值的变量（如本例的 what 和 other）会活得比 match 语句本身更长：只要后面的代码属于同一个作用域（scope）——粗略说就是同一个模块或函数，按本书后面的讲解——它们就可以在 match 退出后被使用。

深度理解

·核心概念：基础 match = "求值头部表达式 → 与各 case 模式自上而下逐一比较 → 第一个命中者执行并退出"。是通配默认，X | Y 是多值或，as 是捕获赋值。

·底层实现：case'go' 这类字面量模式编译为"与 state 的

值做 `==` 比较"; `case other` 这类裸变量模式编译为"无条件匹配 + 赋值"。| 模式编译为依次尝试多个字面量比较。变量捕获本质是"匹配成功后才赋值", 而不是普通语句按序执行。

·设计原因: 与字典和 `if` 相比, `match` 的价值在于模式即语法: 把"匹配 + 解构 + 赋值"打包成声明式的 `case` 头部, 代码意图一目了然。

·实际问题: 配置解析、命令分发、枚举值处理等"按值分支"场景, `match` 让每个分支的"入口条件"与"出口动作"就近组织。

·初学者误区: 1) `case` 不是"变量捕获"——是专用通配符, 不赋值; 2) 以为 `case` 块之间像 `if` 一样有穿透 (`fall-through`) ——`match` 是"命中即止"; 3) 不知道裸变量 `case` (如 `case other:`) 会匹配一切, 必须放在最后, 否则后面所有 `case` 永远不可达。

代码分析

```
# state
if state == 'go':                # if
    print('green')
elif state == 'stop':
    print('red')
else:
    print('yellow')

colors = dict(go='green', stop='red')  #
print(colors.get(state, 'yellow'))

match state:                      # match3.10+
    case 'go':
```



```
    print('green')
case 'stop':
    print('red')
case _:
    print('yellow')
```

解释器如何处理：match state: 先把 state 的值求出来（压栈保存为"匹配对象"），然后逐个 case 生成匹配测试。对 case 'go'，编译出"对象与常量'go' 相等比较"的指令，不等则跳到下一个 case；对 case，编译为无条件跳转到该块。命中后运行该块的字节码，运行完直接跳到整个 match 之后——没有 fall-through，这一点与 C 的 switch 截然不同（C 的 switch 需要在每个分支末尾写 break，否则会穿透进下一个分支）。

设计思想：match 把"选择"从"比较"升级为"模式匹配"：case 头部不再只是条件，而是"形状的样板"。对新手而言，先掌握"字面量 + | + + as + 裸变量"这五个基础形态就够用了。

12.4.2 实战对比：match 与 if (Match versus if live)

英文原文

As a live match example, the following maps statements to categories and assigns some variables along the way for later use; we'll get formal about the for loop used here in the next chapter:

```
>>> for stmt in ['if', 'while', 'try']:
...     match stmt:
...         case 'if' | 'match':
...             print('logic')
...         case 'for' | 'while' as which:
...             print('loop')
...         case other:
...             print('tbd')
...
logic
loop
tbd
>>> which, other
('while', 'try')
```

The equivalent if must assign the variables explicitly—though it requires less indentation, this example is artificial, and the if can handle more complex logic than the match cannot; it's not just a multiple-choice tool:

```
>>> for stmt in ['if', 'while', 'try']:
...     if stmt in ['if', 'match']:
...         print('logic')
...     elif stmt in ['for', 'while']:
...         which = stmt
...         print('loop')
...     else:
...         other = stmt
...         print('tbd')
...
...same results...
```

Watch for more basic match statements to show up later in this book (e.g., in Chapters 25, 30, and 38). Though it cannot be used to code general logic like if,

match can make multiple-choice selection explicit in its basic form. Its extension to the structural pattern matching of the next section, however, is more difficult to rationalize—but you'll have to read on to judge for yourself.

中文翻译

作为一个现场 match 示例，下面把语句名映射到类别，并顺路给后面要用的变量赋值；这里用到的 for 循环要到下一章才会正式讲解：

```
>>> for stmt in ['if', 'while', 'try']:
...     match stmt:
...         case 'if' | 'match':
...             print('logic')
...         case 'for' | 'while' as which:
...             print('loop')
...         case other:
...             print('tbd')
...
logic
loop
tbd
>>> which, other
('while', 'try')
```

等价的 if 必须显式地给变量赋值——虽然它需要的缩进更少，但这个例子是人为构造的，而且 if 能处理 match 处理不了的更复杂逻辑；它不只是多选工具：

```
>>> for stmt in ['if', 'while', 'try']:
...     if stmt in ['if', 'match']:
...         print('logic')
...     elif stmt in ['for', 'while']:
...         which = stmt
...         print('loop')
...     else:
...         other = stmt
...         print('tbd')
...
.....
```

留意本书后面会出现更多基础 match 语句（例如第 25、30、38 章）。虽然 match 不能像 if 那样编写任意逻辑，但它能在基础形态下让多选逻辑显式化。然而，它扩展到下一节“结构模式匹配”的部分则更难论证其价值——但你必须读下去，才能自己做出判断。

深度理解

- 核心概念：本例对比了 match 与 if 的“赋值副作用”差异：match 的捕获变量 (which、other) 是匹配的副产品，自动赋值且存活于 match 之后；if 则必须手动赋值。
- 底层实现：捕获赋值发生在匹配成功那一刻：解释器执行匹配指令时把被匹配对象写入 which/other 所在的局部变量槽。case other: 是“永远匹配 + 无条件赋值”，等效于把 stmt 赋给 other 再执行块。
- 设计原因：match 把“条件判断”和“结果提取”合二为一——这正是结构模式匹配的卖点，哪怕在基础形态下已经初见端倪。
- 实际问题：解析类代码（把输入分类并提取字段）是这种

"匹配即提取"模式的主场。

·初学者误区：1) 以为 which 在没匹配到时也有值——只有命中的 case 才会赋值，否则引用它会 NameError；2) 以为 match 能替代 if 的全部功能——if stmt in [...] 这种"集合成员"判断用 match 只能靠 | 穷举字面量。

代码分析

```
>>> for stmt in ['if', 'while', 'try']:
...     match stmt:
...         case 'if' | 'match':          #
...             print('logic')
...         case 'for' | 'while' as which: # stmt whi...
...             print('loop')
...         case other:                   # other
...             print('tbd')
...
logic
loop
tbd
>>> which, other          # match ('while', 'try')
('while', 'try')
```

解释器如何处理：第一轮循环 stmt = 'if'，与 case 'if' | 'match' 匹配成功，输出 logic，match 立即结束，which/other 未被赋值。第二轮 stmt = 'while'，前两个 case 的 | 测试都不命中，第三个 case 做"捕获赋值"：把 'while' 写入 which，输出 loop。第三轮 stmt = 'try'，落入 case other，把 'try' 写入 other。循环结束后，which 和 other 仍驻留在函数/模块的局部符号表里，所以交互提示符下 which, other 返回 ('while', 'try')。捕获变量的生命周期从"匹配成功"那一刻开始，一直持续到作用域结束——这

是 match 与 if 最本质的差异之一。

设计思想：match 的"匹配即绑定"让"判断"与"取数据"一步完成，省掉了 if 版本里 which = stmt 这样的显式赋值行。代价是：只有命中时才有赋值，未命中路径上的变量是"未定义"的——这是所有声明式匹配语言共同的行为。

12.4.3 高级 match 用法 (Advanced match Usage)

英文原文

As noted, beyond its basic level shown so far, match becomes too complex to cover usefully here. In this guise, it goes well beyond multiple-choice logic, to define a language of its own for extracting components of structured objects. As a very brief survey, this over-caffeinated match treats case values as patterns, which may be:- Literal patterns: a literal X matches the same value, by equality or identity- Wildcard patterns: matches anything, but the value is not assigned to - Capture patterns: variable X matches anything, and will be assigned to it- Or patterns: X | Y | Z matches patterns X or Y or Z, stopping at the first match- As patterns: X as Y matches pattern X, and assigns the matched value to Y- Additional patterns that match sequences, mappings, attributes, and instances by structure As an artificial (if frightening) example, Example 12-1 demos literal, sequence, and mapping patterns. Its

[...] and (...) patterns both match any sequence and are interchangeable; its {...} matches a mapping; and its single or names in patterns collect unmatched parts of a sequence or mapping, respectively. Using a in this context is similar to Chapter 11's extended-unpacking assignments, though here is unique, not all parts of a pattern must be variables, and assignment to starred names is really a side effect of a Boolean test for a match (pasters beware: blank lines added for readability here won't work in a REPL, and [...] patterns preclude (...); run from a file and experiment freely).

Example 12-1. matchdemo.py

```
# state = 1 or
# [1, 2, 3] or [0, 2, 3] or (1, 2, 3) or (0, 2, 3) or
# dict(a=1, b=2, c=3) or dict(a=0, b=2, c=3) or other
match state:
    case 1 | 2 | 3 as what:           # Match integer literals, w...
        print('or', what)
    case [1, 2, what]:               # Match sequence (1), what ...
        print('list', what)
    case [0, *what]:                 # Match sequence (0), what ...
        print('list', what)
    case {'a': 1, 'b': 2, 'c': what}: # Match mapping, what = 3
        print('dict', what)
    case {'a': 0, **what}:            # Match mapping, what = {'b...
        print('dict', what)
    case (1, 2, what):               # Match sequence: same as [...
        print('tuple', what)
    case (0, *what):                 # Match sequence: same as [...
        print('tuple', what)
    case _ as what:                  # Match all other, what = o...
        print('other', what)
```

Subtly, the [...], {...}, and (...) patterns in this code's case headers are not normal object literals. They're really special-case syntax forms that contain nested patterns, many of which just happen to be literal patterns here. Nested patterns can also be capture patterns (possibly after at most one or), and may use | ors, or wildcards. The case header [a, 2 | 3,], for example, is a valid sequence pattern, but has little to do with list literals. Attribute and instance patterns require knowledge you haven't yet gained (a recurring theme in Python today), but they check for attribute values and inheritance-tree membership, and some of their components are nested patterns again, which may be literals, captures, and more. As a preview—which you can revisit after reading Part VI:

```
class Emp:
    def __init__(self, name): self.name = name
pat = Emp('Pat')                # pat.name becomes 'Pat': see Part...
# state = 'Pat' or pat
match state:
    case pat.name as what:      # Match object's attribute, what =...
        print('attr', what)
    case Emp(name=what):        # Match an Emp instance, what = 'P...'
        print('instance', what)
```

And if that's not already overkill for your multiple-choice logic needs, parentheses may be used around any pattern for grouping (as in general expressions); nested structures match recursively as they do in sequence assignment; and each case header can also end with an optional guard expression introduced by an `if`

f (after the optional as capture), which must be true for the case to be selected and its code block run:

```
state = ((1, 2), 3)
guard1 = True                                # a=1, b=3 if True; a=(1, 2) if F...
match state:
    case ((a, 2), b) if guard1: # Match+run only if guard1 is True
        print(f'case1 {a=} {b=}')
    case (a, 3) as what:         # Reached only if guard1 is False
        print(f'case2 {a=} {what=}')
    case [a, (3 | 4)] as what if guard1:
        print(f'case3 {a=} {what=}')
```

All of which seems a blizzard of functionality to address usage that's overwhelmingly straightforward. Hence, this is where this book's match coverage must stop short for space (and mercy). For more on advanced roles of match, including all the gory details of its special-case patterns, consult Python's online resources. Before you do, though, ponder just for a moment on the fact that Python was used successfully for three decades and rose to the top of the programming-languages heap without a match statement. Saddling the language with yet another convoluted subdomain that obviates a trivial amount of code might owe at least as much to hubris as user need. That said, match may be useful in simpler roles, though you'll still need to choose between if, dictionaries fetches, and match when coding multiple-choice logic. While you should generally strive to avoid doing something just because you've done it in other languages (you're now using Python, after all), such choices are yours to make

中文翻译

如前所述，超出我们目前展示的基础层面，`match` 就变得过于复杂，难以在这里有效展开。在这种形态下，它远不止多选逻辑，而是定义了一套自己的“语言”，用于提取结构化对象的组件。作为一次非常简短的巡礼，这种“过度亢奋”的 `match` 把 `case` 的值当作模式（pattern）处理，模式可以是：

- 字面量（Literal）模式：字面量 `X` 通过相等性或同一性匹配同样的值
- 通配符（Wildcard）模式：匹配任何东西，但值不会被赋给
- 捕获（Capture）模式：变量 `X` 匹配任何东西，并且值会被赋给 `X`
- 或（Or）模式：`X | Y | Z` 匹配模式 `X`、`Y` 或 `Z`，停在第一个匹配处
- `as` 模式：`X as Y` 匹配模式 `X`，并把匹配到的值赋给 `Y`

还有按结构匹配序列（sequence）、映射（mapping）、属性（attribute）和实例（instance）的额外模式作为一个人工构造（如果还算吓人）的例子，示例 12-1 演示了字面量、序列和映射模式。它的 `[...]` 和 `(...)` 模式都能匹配任何序列，且二者可互换；`{...}` 匹配映射；模式中的单个或名字分别收集序列或映射中未匹配的部分。在这里使用类似于第 11 章的扩展解包赋值，不过这里的是特有的、并非模式的所有部分都必须是变量、而且对带星名字的赋值实际上是“匹配与否的布尔测试”的副作用（粘贴者注意：这里为可读性加的空行在 REPL 中无效，而且 `[...]` 模式会排斥 `(...)`；请从文件运行并自由实验）。

示例 12-1. `matchdemo.py`

```

# state = 1
# [1, 2, 3] [0, 2, 3] (1, 2, 3) (0, 2, 3)
# dict(a=1, b=2, c=3) dict(a=0, b=2, c=3)
match state:
    case 1 | 2 | 3 as what:                # what = 1
        print('or', what)
    case [1, 2, what]:                      # (1)what = 3
        print('list', what)
    case [0, *what]:                        # (0)what = [2, 3]
        print('list', what)
    case {'a': 1, 'b': 2, 'c': what}:      # what = 3
        print('dict', what)
    case {'a': 0, **what}:                  # what = {'b': 2, 'c':...
        print('dict', what)
    case (1, 2, what):                      # [1, 2, what]
        print('tuple', what)
    case (0, *what):                        # [0, *what]
        print('tuple', what)
    case _ as what:                         # what = other
        print('other', what)

```

微妙之处在于：这段代码 case 头部中的 [...]、{...} 和 (...) 模式并不是普通的对象字面量。它们其实是“特例语法”形式，里面包含嵌套的模式，这里的很多嵌套模式碰巧都是字面量模式。嵌套模式也可以是捕获模式（可能跟在一个或之后），还可以使用 | 或运算、通配符。例如 case 头部 [a, 2 | 3,] 是一个合法的序列模式，但与列表字面量没什么关系。属性（attribute）和实例（instance）模式需要你尚未掌握的知识（这是当今 Python 里反复出现的主题），但它们检查的是属性值和继承树成员关系，而且它们的一些组件又是嵌套模式，可以是字面量、捕获等等。作为一个预告——你可以在读完第六部分之后再回来看它：

```

class Emp:
    def __init__(self, name): self.name = name
pat = Emp('Pat')                # pat.name  'Pat'
# state = 'Pat' pat
match state:
    case pat.name as what:      # what = 'Pat'
        print('attr', what)
    case Emp(name=what):        # Emp what = 'Pat'
        print('instance', what)

```

如果这对你的多选逻辑需求来说还不够过度设计的话：任何模式外面都可以加圆括号来分组（与一般表达式相同）；嵌套结构会像序列赋值那样递归匹配；而且每个 case 头部末尾还可以带一个可选的守卫（guard）表达式，由 if 引入（跟在可选的 as 捕获之后），它必须为真，该 case 才会被选中并运行其代码块：

```

state = ((1, 2), 3)
guard1 = True                    # Truea=1, b=3 Falsea=(1,...
match state:
    case ((a, 2), b) if guard1: # guard1 True
        print(f'case1 {a=} {b=}')
    case (a, 3) as what:         # guard1 False
        print(f'case2 {a=} {what=}')
    case [a, (3 | 4)] as what if guard1:
        print(f'case3 {a=} {what=}')

```

所有这些看起来像是一场功能风暴，而它要解决的用法却压倒性地简单直接。因此，本书对 match 的覆盖到这里就必须止步了——为了篇幅（也为了仁慈）。关于 match 高级角色的更多内容，包括其特例模式的全部细节，请查阅 Python 的在线资源。不过，在你查阅之前，请稍微想一想这个事实：Python 在没有 match 语句的情况下成功使用了三十年，并登上了编程语言之颠。给这门语言再背负一个“消除了一丁点代码量的复杂子领域”，可能至少有同样多的“

傲慢"成分，而不只是用户需求。话虽如此，match 在更简单的角色上可能是有用的，不过你在编写多选逻辑时，仍然需要在 if、字典取值和 match 之间做出选择。虽然你通常应该尽量避免"仅仅因为在其他语言里这么做就照搬"（毕竟你现在用的是 Python），但这些选择权在你自己手里。

深度理解

·核心概念：高级 match = 一套迷你语言：字面量/通配符/捕获/或/as/序列/映射/属性/实例模式 + 守卫。核心思想是"匹配即解构"：一次匹配同时完成判断、拆包、赋值三件事。

·底层实现：每种模式在编译期被翻译成特定的匹配字节码。序列模式 [1, 2, what] 会先检查 state 是不是序列（非字符串的 iterable）、长度是否为 3、逐元素比较字面量、把第 3 个元素赋给 what。映射模式 {'a': 1, what} 先检查是否 dict、键'a'的值是否为 1，再把剩余条目收集到 what。守卫 if guard1 在模式匹配成功后、执行块之前多一次布尔跳转。属性模式 pat.name 在编译期解析为对象属性访问；实例模式 Emp(name=what) 做 isinstance 检查（继承树成员测试）加关键字捕获。

·设计原因：PEP 634/635/636 的设计目标来自函数式语言（ML、Haskell、Scala）的模式匹配传统：让"对结构化数据的形状进行分流"成为语言一级的声明式语法，替代手写的 isinstance + 索引/解包 + 赋值序列。代价是语法复杂度陡增。

·实际问题：真正的应用场景集中在：解析器（AST/语法

树节点分发)、JSON/配置文件的结构化提取、协议报文解码。日常业务逻辑很少需要。

·初学者误区：1) 把 [...] 模式当成列表字面量去"创建"列表——它只做匹配不创建对象；2) case {'a': 0, what} 里 what 收集的是"剩余键值对"而不是"所有项"；3) 忘记守卫失败时该 case 不算命中，会继续尝试下一个 case；4) case as what 中依然是通配符（不绑定），what 才是绑定名——和 as 各司其职。

代码分析

```
# 12-1 "" state
match state:
    case 1 | 2 | 3 as what:          # +asstate 1/2/3 w...
        print('or', what)
    case [1, 2, what]:              # 3 12
        print('list', what)
    case [0, *what]:                # 0 what
        print('list', what)
    case {'a': 1, 'b': 2, 'c': what}: #
        print('dict', what)
    case {'a': 0, **what}:           # a 0 what
        print('dict', what)
    case (1, 2, what):              # / [1, 2, what]
        print('tuple', what)
    case (0, *what):
        print('tuple', what)
    case _ as what:                 # what = state
        print('other', what)
```

解释器如何处理：以 state = [0, 2, 3] 为例。第 1 个 case：先尝试 1 | 2 | 3 与 [0,2,3] 做相等比较——[0,2,3] == 1 为假，或模式失败，as 捕获不发生。第 2 个 case：检

查 `[0,2,3]` 是否序列（是）、长度是否 3（是）、`[0]` 与 1 比较（假），失败。第 3 个 case：首元素 0 相等，星号捕获把 `[2, 3]` 收集给 `what`，匹配成功，运行块输出 `list [2, 3]`，整个 `match` 结束。注意 `{...}` 映射模式不会与列表误配——解释器先做类型检查（是否 `dict` 实例）再逐键匹配，这正是“结构”匹配的含义：模式描述的是“形状”而不是“具体值”。守卫 case 则是在形状匹配后、块执行前多一次 `POPJUMPIFFALSE` 式的判断。

设计思想：作者用“blizzard of functionality”（功能风暴）形容它，并提醒：Python 三十年没有 `match` 也登顶了语言排行榜。这句话的深意是——语言特性不是越多越好，够用、简单、可读才是 Python 的立身之本。对学习者：基础形态拿来即用，高级形态按需查阅手册。

12.5 Python 语法回顾 (Python Syntax Revisited)

英文原文

Python's syntax model was introduced in Chapter 10. Now that we're stepping up to larger statements like `if` and `match`, this section reviews and expands on the syntax ideas introduced earlier. In general, Python has a simple, statement-based syntax. Among its highlights:- Statements execute one after another, until you say otherwise. Python normally runs statements in a file or nested block in order from first to last, but st

statements like if (as well as loops and exceptions) cause the interpreter to jump around in your code. Because Python's path through a program is called the control flow, statements such as if that affect it are often called control-flow statements.

- Block and statement boundaries are detected automatically. As we've seen, there are no braces or "begin/end" delimiters around blocks of code in Python; instead, Python uses the indentation of statements under a header to group the statements in a nested block. Similarly, Python statements are not normally terminated with semicolons; rather, the end of a line usually marks the end of the statement coded on that line. As special cases you'll meet later, statements can both span lines and be combined on a line when it's useful.
- Compound statements = header + ":" + indented statements. All Python compound statements—those with nested statements—follow the same pattern: a header line terminated with a colon, followed by one or more nested statements, usually indented under the header. The indented statements are called a block (or sometimes, a suite). In the if statement, the elif and else are part of the if, but they are also header lines with nested blocks of their own. As a special case, blocks can be on the same line as the header if they are not compound.
- Blank lines, spaces, and comments are usually ignored. Blank lines are both optional and ignored in files (but not at the interactive prompt, when they termina

te compound statements). Spaces inside statements and expressions are almost always ignored (except in string literals, and when used for indentation). Comments are always ignored: they start with a # character (not inside a string literal) and extend to the end of the current line. Docstrings are ignored but are saved and displayed by tools. Python supports an additional comment form called documentation strings (docstrings for short), which, unlike # comments, are retained at runtime for inspection. Docstrings are simply strings that show up at the top of program files and some statements. Their content is ignored, but they are attached to objects and may be displayed with tools covered later in this book. For most Python newcomers, the lack of the braces and semicolons used to mark blocks and statements in many other languages seems to be the most novel syntactic feature of Python, so let's explore what this means in more depth.

中文翻译

Python 的语法模型在第 10 章已经介绍过。现在我们要升级到 if 和 match 这样更大的语句，本节将回顾并扩展之前引入的语法概念。总的来说，Python 拥有简单、基于语句（statement-based）的语法。其中要点如下：- 语句一条接一条执行，除非你另有安排。Python 通常按从第一条到最后一条的顺序运行文件或嵌套块中的语句，但 if 这类语句（以及循环和异常）会让解释器在你的代码中跳转。因为 Python 在程序中的行进路径被称为控制流（control flow

），所以 if 这类影响控制流的语句常被称为控制流语句（control-flow statement）。- 块和语句的边界自动检测。正如我们所见，Python 中代码块周围没有花括号或 "begin/end" 定界符；相反，Python 用"头部行下面的语句缩进"来把语句分组为嵌套块。同样，Python 语句通常不用分号结尾；而是用"行尾"通常标记该行语句的结束。作为特殊情况（后面会见到），语句既可以跨行，也可以在一行内合并，只要有用。- 复合语句 = 头部 + ":" + 缩进的语句。所有 Python 复合语句（compound statement）——即带有嵌套语句的那些——都遵循同一模式：以冒号结尾的头部行，后面跟一条或多条嵌套语句，通常缩进在头部之下。缩进的语句被称为块（block）（有时也叫 suite）。在 if 语句中，elif 和 else 是 if 的一部分，但它们本身也是头部行，各自带有自己的嵌套块。作为特例，如果块不是复合语句，它可以与头部放在同一行。- 空行、空格和注释通常被忽略。空行在文件中既可省略也会被忽略（但在交互提示符下除外——那里空行会终止复合语句）。语句和表达式内部的空格几乎总被忽略（字符串字面量内、以及用于缩进时除外）。注释总是被忽略：它们以 # 字符开头（字符串字面量内不算），一直延伸到当前行尾。- Docstring（文档字符串）被忽略，但会被保存并被工具显示。Python 支持一种附加的注释形式，称为文档字符串（docstring），与 # 注释不同，它在运行时被保留以供检查。Docstring 只是出现在程序文件和某些语句顶部的字符串。它们的内容被忽略，但会挂到对象上，可以用本书后面介绍的工具显示。对大多数 Python 新手来说，缺少其他语言用来标记块和语句的花括号和分号，似乎是 Python 最新奇的语法特征，所以让我们更深入地探索这意味着什么。

深度理解

·核心概念：Python 语法模型的五个要点——顺序执行、缩进定界、冒号头部、忽略空白与注释、docstring 特例。合起来就是"可读性优先"的语法设计。

·底层实现：CPython 的词法/语法阶段 (Tokenizer/Parser) 在编译期逐行统计缩进，遇到缩进增加就生成 INDENT 记号、缩进减少生成 DEDENT 记号，由它们构造语句的嵌套树 (AST)，再编译为字节码。"顺序执行"在字节码层面就是指令逐条线性执行，if 的跳转指令改变 PC (程序计数器)。

·设计原因：去掉定界符，迫使代码"以一致的缩进书写"，从根源上消除"括号错位"这类经典 bug，并让任何编辑器打开代码都是同样的结构。

·实际问题：正因为边界自动检测，粘贴代码、混用 tab/空格、缩进错位都会立刻产生 IndentationError——这既是保护的栅栏，也是新手最常撞的墙。

·初学者误区：1) 以为 # 注释会被解释器优化掉——其实编译期就完全移除了；2) 以为 docstring 是普通注释——它是表达式语句，运行时会真实求值并挂载到 doc 属性上；3) 在 REPL 里敲空行以为"没影响"——它会终止复合语句。

12.5.1 块定界符：缩进规则 (Block Delimiters: Indentation Rules)

英文原文

As introduced in Chapter 10, Python detects block boundaries automatically, by line indentation—that is, the empty space to the left of your code. This section is a rehash of the rules, with a few more details sprinkled in along the way. In short, all statements indented the same distance to the right belong to the same block of code. In other words, the statements within a block line up vertically, as in a column. The block ends when a lesser-indented line or the end of the file is encountered (or you enter a blank line in a REPL), and more deeply nested blocks are simply indented further to the right than the statements in the enclosing block. For instance, Figure 12-1 demonstrates the block structure of the following code:

```
x = 1
if x:
    y = 2
    if y:
        print('block2')
    print('block1')
print('block0')
```

This code contains three blocks: the first (the top-level code of the file) is not indented at all, the second (within the outer if statement) is indented four spaces, and the third (the print statement under the nested if) is indented eight spaces. Top-level (unnested) code must start in column 1, but nested blocks can start in

any column; indentation may consist of any number of spaces and tabs, as long as it's the same for all the statements in a given single block. That is, Python doesn't care how you indent your code; it only cares that it's done consistently. Four spaces or one tab per indentation level are common conventions, but there is no absolute standard or rule in the Python world. Indenting code is quite natural in practice. For example, the following fully frivolous code snippet demonstrates common indentation errors in Python code, which are easy to spot because they're visually askew:

```
x = 'Hack'                                # Error: first line indented
if 'tho' in 'python':
    print(x * 8)
    x += 'More!'                          # Error: unexpected indentation
    if x.endswith('re!'):                 # Error: inconsistent indentation
        x *= 2
    print(x)
```

The properly indented version of this code looks like the following—even for an artificial example like this, proper indentation makes the code's intent much more apparent:

```
x = 'Hack'
if 'tho' in 'python':
    print(x * 8)                          # Prints 8 Hack
    x += 'More!'
    if x.endswith('re!'):
        x *= 2
    print(x)                             # Prints HackMore!HackMore!
```

It's important to know that the only major place in Py

thon where whitespace matters is where it's used to the left of your code, for indentation; in most other contexts, space can be coded or not. However, indentation is really part of Python syntax, not just a stylistic suggestion: all the statements within any given single block must be indented to the same level, or Python reports a syntax error. This is intentional—because you don't need to explicitly mark the start and end of a nested block of code, some of the syntactic clutter found in other languages is unnecessary in Python. As described in Chapter 10, making indentation part of the syntax model also enforces consistency, a crucial component of readability in structured programming languages like Python. In Python's syntax, the indentation of each line of code unambiguously tells readers what it is associated with. This uniform and consistent appearance in turn makes Python code easier to maintain and reuse. In the end, indentation is easier than you may think. Consistently indented code always satisfies Python's rules, and most text editors (including IDLE) make it easy to follow Python's model by automatically indenting as you type.

中文翻译

正如第 10 章所介绍的，Python 通过行的缩进 (indentation) ——即代码左侧的空格——自动检测块边界 (block boundary)。本节是对规则的复习，顺带补充一些细节。简而言之，所有向右缩进相同距离的语句都属于同一个代码

块。换句话说，块内的语句像一列一样垂直对齐。当遇到缩进更少的行或文件结尾时（或在 REPL 中输入空行时），块就结束了；更深层的嵌套块只是比其外层块中的语句缩进得更靠右。例如，图 12-1 演示了以下代码的块结构：

```
x = 1
if x:
    y = 2
    if y:
        print('block2')
    print('block1')
print('block0')
```

这段代码包含三个块：第一个（文件的顶层代码）完全不缩进；第二个（外层 if 语句内）缩进四个空格；第三个（嵌套 if 下的 print 语句）缩进八个空格。顶层（未嵌套的）代码必须从第 1 列开始，但嵌套块可以从任何列开始；缩进可以由任意数量的空格和 tab 组成，只要同一块中的所有语句一致即可。也就是说，Python 不在乎你怎么缩进，只在乎缩进是否一致。每个缩进层级用四个空格或一个 tab 是常见约定，但在 Python 世界中没有绝对标准或规则。实践中缩进代码是很自然的事。例如，下面这段完全无聊的代码片段演示了 Python 代码中常见的缩进错误——它们很容易发现，因为视觉上就歪歪扭扭：

```
x = 'Hack'                                #
if 'tho' in 'python':
    print(x * 8)
    x += 'More!'                           #
    if x.endswith('re!'):                  #
        x *= 2
    print(x)
```

这段代码正确缩进的版本如下——即便是这样一个人为的

例子，正确的缩进也让代码意图清晰得多：

```
x = 'Hack'
if 'tho' in 'python':
    print(x * 8)                # 8 Hack
    x += 'More!'
    if x.endswith('re!'):
        x *= 2
    print(x)                    # HackMore!HackMore!
```

要知道，Python 中空白"重要"的主要场合只有代码左侧的缩进；在大多数其他上下文中，空格可有可无。然而，缩进确实是 Python 语法的一部分，而不仅仅是风格建议：任何给定块内的所有语句必须缩进到同一层级，否则 Python 会报告语法错误。这是故意的——因为你不需要显式标记嵌套代码块的开始和结束，其他语言中的一些语法杂物在 Python 中就是多余的。如第 10 章所述，让缩进成为语法模型的一部分也强制了一致性——这是 Python 这类结构化编程语言可读性的关键要素。在 Python 的语法中，每一行代码的缩进都毫不含糊地告诉读者它关联到什么。这种统一一致的观感反过来让 Python 代码更易维护、更易复用。归根结底，缩进比你想象的容易。一致缩进的代码永远满足 Python 的规则，大多数文本编辑器（包括 IDLE）都会在你输入时自动缩进，让你轻松跟上 Python 的模型。

深度理解

- 核心概念：缩进即语法。同一缩进 = 同一块；块结束于"缩进变少"或文件末尾；嵌套 = 更深的缩进。顶层代码必须从第 1 列开始。

- 底层实现：Tokenize 阶段，解释器把缩进换算成虚拟的 INDENT/DEDENT 记号，Parser 依据它们构建语法树。

缩进不一致（如同一个块里 4 空格和 2 空格混用）会直接抛 `IndentationError`——这是语法错误，程序根本不会开始运行。

·设计原因：缩进参与语法，等价于强制全员统一排版。Guido 在 1989 年设计时的理由：既然代码反正都要缩进，不如让语言“强制执行”，顺带消灭定界符相关的低级错误。

·实际问题：粘贴代码（论坛/微信里的代码可能带前导空格）、tab 与空格混用、编辑器配置不同，都会引发缩进错误。python -m tabnanny 这类工具可以检测混用。

·初学者误区：1) 以为“缩进只是好看”——它是语法错误的高发源头；2) 在 REPL 里误用 tab 缩进（REPL 对 tab 的换算和编辑器不同）；3) 顶层代码从列 1 之外开始（比如整段多缩进了两格）——会报 `IndentationError: unexpected indent`。

代码分析

```
x = 1
if x:
    y = 2
    if y:
        print('block2')    # if 8
        print('block1')    # if 4
print('block0')            # 0
```

解释器如何处理：Tokenizer 逐行读取，统计行首空白数量。第 1 行缩进 0；第 3 行缩进 4 → 发出 INDENT 记号（进入块 2）；第 5 行缩进 8 → 再次 INDENT（进入块 3）；第 6 行缩进 4 → DEDENT（离开块 3）；第 7 行缩

进 0 → DEDENT (离开块 2) 。Parser 靠这对记号确定嵌套深度，最终生成 AST: if x 的子节点是 y=2; if y: ... 组成的块。执行时，先求 x (为真) ，进入外层块；再求 y (为真) ，执行 print('block2'); 退出内层块后执行 print('block1'); if 语句结束，执行顶层 print('block0')。输出顺序 block2 / block1 / block0 印证了"块 = 跳转的目标区间"。

设计思想：把排版规范提升为语法，Python 是唯一"强制执行缩进"的主流语言。这带来的不仅是美观，还有代码结构的强制可见性——读任何人的 Python 代码，块结构都一目了然。

12.5.2 避免混用 tab 和空格 (Avoid mixing tabs and spaces)

英文原文

That said, there's one rule of thumb you should know : although you can use spaces or tabs to indent, it's usually not a good idea to mix the two within a block —use one or the other. Technically, tabs count for enough spaces to move the current column number up to a multiple of 8, and your code will work if you mix tabs and spaces consistently. However, mixing tabs and spaces makes code difficult to read and change completely apart from Python's syntax rules—tabs may look very different in the next programmer's editor than they do in yours. For these reasons, Python issu

es an error when a script mixes tabs and spaces for in dentation inconsistently within a block (that is, in a way that makes it dependent on a tab's equivalent in spaces). So don't do that: when in Python do as Pythoners do and use consistent indentation instead of block delimiters.

中文翻译

话虽如此，有一条经验法则你应该知道：虽然你可以用空格或 tab 缩进，但在一个块内混用两者通常不是好主意——二选一。严格来说，tab 按“足够把当前列号推进到 8 的倍数”的空格数计算，如果你一致地混用 tab 和空格，代码也能工作。然而，混用 tab 和空格会让代码难以阅读和修改——这与 Python 的语法规则无关：tab 在下一个程序员的编辑器里可能和在你这里看起来完全不同。由于这些原因，当脚本在一个块内不一致地混用 tab 和空格做缩进时（即缩进结果取决于“tab 等于多少空格”时），Python 会报错。所以不要这样做：在 Python 中就要像 Pythoners 那样，使用一致的缩进而不是块定界符。

深度理解

·核心概念：同一块内 tab 与空格混用会被 Python 判为语法错误 (TabError: inconsistent use of tabs and spaces in indentation) 。

·底层实现：tab 的实际宽度等于“推进到下一个 8 的倍数”的空格数。如果编辑器 A 显示 tab=4 格、编辑器 B 显示 tab=8 格，同一行代码的“视觉缩进”完全不同，但 Python

n 只按"列号对 8 取整"解释——于是不同行的实际缩进深度可能悄悄不同，轻则报错、重则形成语义错误的块结构。

- 设计原因：Python 无法知道"你希望 tab 是几格"，唯一的确定性方案就是：混合使用时直接报错，逼你统一。

- 实际问题：从网页、Word、聊天软件复制代码到 IDE 时经常混入 tab；PEP 8 官方建议使用 4 空格。现代编辑器（VS Code、PyCharm）默认"输入 Tab 键 = 插入 4 空格"，可彻底避开此坑。

- 初学者误区：以为"语法能过就没事"——即使碰巧通过了，代码也可能在你同事的机器上变成完全不同的结构。

12.5.3 语句定界符：行与续行 (Statement Delimiters: Lines and Continuations)

英文原文

While blocks are indented, a statement in Python normally ends at the end of the line on which it appears. When a statement is too long to fit on a single line, though, a few special rules may be used to make it span multiple lines:- Statements may span multiple lines if you're continuing an "open pair." The code of a statement can always be continued on the next line if it's enclosed in a (), {}, or [] pair. For instance, expressions in parentheses and dictionary and list literals can span any number of lines; the statement doesn't end until the end of the line containing the closing part o

f the pair (a), }, or])). Continuation lines—lines 2 and beyond of the statement—can start at any indentation level, but it's best to align vertically for readability in some fashion.- Statements may span multiple lines if they end in a backslash. Though best used as a fallback option, if a statement needs to span multiple lines, you can also add a backslash—a \ not embedded in a string literal or comment—at the end of the prior line to indicate you're continuing on the next line. Because you can also continue by adding parentheses around most constructs, backslashes are rarely used to day. This approach is also error-prone: accidentally forgetting a \ may generate a syntax error or cause the next line to run independently.- Statements may be combined if separated with a semicolon. Though uncommon, you can terminate a statement with a semicolon. This is sometimes used to squeeze more than one statement onto a single line by separating them with semicolons, but this works only when the combined statements are not compound.- Statements may contain multiline strings. As we learned in Chapter 7, triple-quoted string blocks are designed to span multiple lines normally. We also learned in Chapter 7 that adjacent string literals are implicitly concatenated; when this is used in conjunction with the open-pairs rule mentioned earlier, wrapping this construct in parentheses allows it to span multiple lines.

中文翻译

块是靠缩进的，而 Python 中的一条语句通常在其所在行的行尾结束。不过，当语句太长、一行放不下时，可以用几条特殊规则让它跨多行：- 如果延续一个"打开的括号对"，语句可以跨多行。只要语句代码被包在 ()、{} 或 [] 对里，就总可以续行。例如，圆括号中的表达式、字典和列表字面量可以跨任意多行；语句要到"包含括号对右半部分的那一行的行尾"才结束（即)、} 或]）。续行（continuation line）——语句的第 2 行及以后——可以从任何缩进层级开始，但最好以某种方式垂直对齐以利阅读。- 如果以反斜杠结尾，语句可以跨多行。虽然最好只作为备选方案，但如果语句需要跨多行，你也可以在前一行的行尾加一个反斜杠——一个不在字符串字面量或注释中的 \——表示下一行继续。因为大多数结构加个括号就能续行，反斜杠如今很少使用。这种方式也容易出错：不小心漏掉 \ 可能产生语法错误，或导致下一行被独立执行。- 用分号分隔时，语句可以合并。虽然不常见，但你可以用分号结束一条语句。有时用分号把多条语句挤到同一行，但这只在被合并的语句不是复合语句时才有效。- 语句可以包含多行字符串。正如第 7 章所学的，三引号字符串块天生设计成可跨多行。第 7 章我们还学过：相邻的字符串字面量会隐式拼接；当这与前面提到的"打开括号对"规则结合时，用圆括号包住这个结构就能让它跨多行。

深度理解

·核心概念：Python 的语句分隔靠"行尾"，但有四个例外：开放的括号对、行尾反斜杠、分号合并、三引号字符串

·底层实现：Tokenize 阶段，Tokenzier 会记录"括号深度"。当括号深度 > 0 时，行尾的换行不会产生 NEWLINE 记号（语句不结束）；括号闭合后第一个行尾才产生 NEWLINE。反斜杠则把换行直接吞掉（续行）。三引号字符串里的换行属于字符串内容，天然不算语句边界。

·设计原因：这是"显式优于隐式"的又一个体现——续行规则必须看得见摸得着（括号是可见的、反斜杠是可见的），而不是像 C 那样"换行永远只是空白"。

·实际问题：长条件表达式、长列表/字典/函数调用是续行的主要场景。社区惯例：一律用括号续行，不用反斜杠。

·初学者误区：1) 在 if 条件长时写反斜杠——括号方案更稳；2) 漏掉反斜杠导致两行被当两条语句执行，产生隐蔽 bug（本章后面专门给了这个例子）；3) 以为分号是必须的——Python 分号完全可选。

12.5.4 特殊语法实例 (Special Syntax Cases in Action)

英文原文

Here's what a continuation line looks like using the open-pairs rule just described. Delimited constructs, such as lists in square brackets, can span across any number of lines:

```
L = ['app',  
    'script',  
    'program']          # Open pairs may span lines
```

This also works for list comprehensions enclosed in `[]`; anything in `()` (tuples, expressions, function argument and headers, and generators expressions); and anything in `{}` (dictionary and set literals and comprehensions). Some of these are tools we'll study in later chapters, but this rule naturally covers most constructs that span lines in practice. If you're accustomed to using backslashes to continue lines, you can in Python, too, but it's not common practice:

```
if a == b and c == d and \  
    d == e and f == g:  
    print('old school')    # Backslashes allow continuations...
```

Because any expression can be enclosed in parentheses, you can usually use the open-pairs technique instead if you need your code to span multiple lines—simply wrap a part of your statement in parentheses:

```
if (a == b and c == d and  
    d == e and e == f):  
    print('new school')    # But parentheses usually do too, and ...
```

In fact, backslashes are generally discouraged, because they're too easy to not notice and too easy to omit altogether. In the following, `x` is assigned 10 with the backslash, as intended; if the backslash is accidentally omitted, though, `x` is assigned 6 instead, and no error is reported (the `+4` is a valid expression statement by itself). In a real program with a more complex as

signment, this could be the source of a very obscure bug:

```
x = 1 + 2 + 3 \
    +4                # Omitting the \ makes this very different!
```

As another special case, Python allows you to write more than one noncompound statement (i.e., statements without nested statements) on the same line, separated by semicolons. Some coders use this form to save program file real estate, but it usually makes for more readable code if you stick to one statement per line for most of your work:

```
x = 1; y = 2; print(x)        # More than one simple statement
```

As covered in Chapter 7, triple-quoted string literals span lines too. In addition, if two string literals appear next to each other, they are concatenated as if a + had been added between them—when used in conjunction with the open-pairs rule, wrapping in parentheses allows this form to span multiple lines. For example, the first of the following inserts newline characters at line breaks and assigns S to '\naaaa\nbbbb\ncccc', and the other two implicitly concatenate and assign S to 'aaaabbbbcccc'; as we also saw in Chapter 7, # comments are ignored in the second form but included in the string in the first, and f-strings require f prefixes even on continuations lines:

```

S = """
aaaa
bbbb
cccc"""

S = ('aaaa'
     'bbbb'          # Comments here are ignored, add \n if needed
     'cccc')

S = (f'{ 'a' * 4}'    # Also makes 'aaaabbbbcccc'
     f'{ 'b' * 4}'    # Use f'' on each f-string part
     r'cccc')        # And ditto for r'' raw-string parts

```

Finally, and also as a review, Python lets you move a compound statement's body up to the header line, provided the body contains just simple (noncompound) statements. You'll most often see this used for simple if statements with a single test and action, as in the interactive loops we coded in Chapter 10:

```

if 1: print('hello')          # Simple statement on header li...

```

With a little effort, you can combine some of these special cases to write Python code that is difficult to read, but it's not generally recommended. As a rule of thumb, try to keep each statement on a line of its own, and indent all but the simplest of blocks. Six months down the road, you'll be happy you did.

中文翻译

下面是使用刚才描述的“打开括号对”规则的续行示例。方括号等带定界符的结构可以跨任意行数：

```
L = ['app',
     'script',
     'program']          #
```

这同样适用于 [] 中的列表推导式；() 中的任何内容（元组、表达式、函数参数和头部、生成器表达式）；以及 {} 中的任何内容（字典和集合字面量及推导式）。其中一些工具我们会在后面的章节学习，但这条规则自然覆盖了实践中大多数跨行结构。如果你习惯用反斜杠续行，Python 里也可以，但这不是常见做法：

```
if a == b and c == d and \
    d == e and f == g:
    print('old school')    # ...
```

因为任何表达式都可以用圆括号包起来，所以当你需要代码跨多行时，通常可以用打开括号对的技术代替——只需把语句的一部分包进圆括号：

```
if (a == b and c == d and
    d == e and e == f):
    print('new school')    #
```

事实上，反斜杠通常不被推荐，因为它们太容易被忽略、也太容易整个漏掉。下面这段代码中，带反斜杠时 x 被赋值为 10（符合预期）；但如果反斜杠被不小心漏掉，x 就会被赋值为 6，而且不会报任何错误（+4 本身是一条合法的表达式语句）。在真实程序中，如果赋值更复杂，这可能成为非常隐蔽 bug 的源头：

```
x = 1 + 2 + 3 \
    +4          # \
```

作为另一个特例，Python 允许你在同一行写多条非复合语句（即不含嵌套语句的语句），用分号分隔。有些程序员用

这种形式节省文件空间，但对你大多数工作来说，坚持一行一条语句通常会让代码更可读：

```
x = 1; y = 2; print(x)          #
```

正如第 7 章所讲的，三引号字符串字面量也可以跨行。此外，如果两个字符串字面量相邻，它们会像中间加了 + 一样被拼接——当与打开括号对规则结合时，用圆括号包住这种结构就能让它跨行。例如，下面第一个写法在换行处插入换行符，把 S 赋为 '\naaaa\nbbbb\ncccc'; 另外两个写法隐式拼接，把 S 赋为 'aaaabbbbcccc'; 如第 7 章所见，# 注释在第二种写法中被忽略，但在第一种写法中会包含在字符串里；而 f-字符串 (f-string) 即使在续行上也要求 f 前缀：

```
S = """
aaaa
bbbb
cccc"""

S = ('aaaa'
     'bbbb'          # \n
     'cccc')

S = (f'{ 'a' * 4}'    # 'aaaabbbbcccc'
     f'{ 'b' * 4}'    # f-string f''
     r'cccc')        # r''
```

最后，也算是一次复习：Python 允许你把复合语句的语句体移到头部行上，前提是语句体只包含简单（非复合）语句。最常见的场景是“单个测试 + 单个动作”的简单 if 语句，就像我们在第 10 章写的交互式循环那样：

```
if 1: print('hello')          #
```

稍微花点心思，你就能把这些特殊案例组合起来，写出难以阅读的 Python 代码，但通常不推荐这么做。作为经验法则：尽量让每条语句独占一行，除了最简单的块之外都保持缩进。六个月后回头看，你会庆幸自己当时这么做了。

深度理解

·核心概念：本节把四条续行/合并规则的用法和坑全部演示了一遍：括号续行（推荐）、反斜杠续行（不推荐，漏写产生隐蔽 bug）、分号合并（谨慎）、相邻字符串自动拼接。

·底层实现：漏反斜杠的 bug 原理： $x = 1 + 2 + 3$ 是一条完整语句， $+4$ 是另一条"表达式语句"（Python 允许裸表达式作为语句，只是结果被丢弃）。解释器不报错， x 悄悄变成 6——这就是"语法合法但语义错误"的典型。

·设计原因：Python 故意不默认"换行即忽略"，而是让续行必须显式（括号可见、反斜杠可见），这样代码的"物理行结构"和"逻辑语句结构"之间没有隐式歧义。

·实际问题：数据库查询字符串、长函数调用参数、f-string 拼接是续行的高频场景；"一行一语句"是 PEP 8 的明确建议。

·初学者误区：1) 以为 `f{'a' 4}` 里内层引号写法随意——f-string 里的引号必须与前缀区分；2) 在 REPL 里粘贴多行 f-string 拼接时把续行提示符 `...` 也粘贴进去；3) 以为"表达式语句"无意义就随便裸写——它会在调试时制造"为什么没输出"的困惑。

代码分析

```
L = ['app',  
    'script',  
    'program']          #  
  
if (a == b and c == d and    #  
    d == e and e == f):  
    print('new school')  
  
x = 1 + 2 + 3 \              # x = 1+2+3+4 = 10  
    +4  
  
x = 1 + 2 + 3                # x = 6  
+4                            # +4  
  
S = ('aaaa'  
    'bbbb'                   # 'aaaabbbbcccc'  
    'cccc')
```

解释器如何处理：打开括号对时，Tokenizer 把换行暂时屏蔽，直到闭合括号所在行末才产生 NEWLINE 记号，因此第 1 个例子是一条语句、一个列表。圆括号版 if 同理。反斜杠续行在词法层直接把 \ 与换行一起吞掉，两行合并为一行再编译；漏写反斜杠时，第 1 行是完整的赋值语句，第 2 行 +4 编译为表达式语句——解释器求值后丢弃结果，无任何错误提示。相邻字符串拼接发生在编译期（常量折叠），('aaaa'bbbb'cccc') 在生成字节码前就已经合并为一个字符串常量，运行时零开销。

设计思想：这些“特殊语法”都是可选项而不是必需的。Python 的态度是：规则给你，选择权也给你——但“一行一语句 + 括号续行”才是社区验证过的默认答案。

12.6 真值回顾 (Truth Values Revisited)

英文原文

The notions of comparison, equality, and truth values were introduced in Chapter 9. Like syntax, though, the `if` is the first statement we've studied that actually uses these tools, so we'll rehash these ideas with additional info. All told, Python's Boolean operators are a bit different from their counterparts in some other languages. In Python:

- All objects have an inherent Boolean true or false value.
- Any nonzero number or nonempty object is true.
- Zero numbers, empty objects, and the special object `None` are considered false.
- Comparisons and equality tests are applied recursively to data structures.
- Comparisons and equality tests return `True` or `False` (custom versions of 1 and 0).
- Boolean `and` and `or` operators return a true or false operand object.
- Boolean operators stop evaluating (short-circuit) as soon as a result is known. The `if` statement takes action on truth values, but Boolean operators are used to combine the results of other tests in richer ways to produce new truth values. More formally, there are three Boolean expression operators in Python:

- `X and Y` — Is true if both `X` and `Y` are true—and returns either `X` or `Y`
- `X or Y` — Is true if either `X` or `Y` is true—and returns either `X` or `Y`
- `not X` — Is true if `X` is false—and returns either `True` or `False`

Here, `X` a

and Y may be any truth value, or any expression that returns a truth value (e.g., an `==` equality test, an `in` membership check, and so on). Boolean operators are typed out as words in Python (instead of C's `&&`, `||`, and `!`), and shouldn't be confused with Python's `&`, `|`, and `^` operators that work on numbers and sets (and dictionaries). Most uniquely, Boolean `and` or operators return a `true` or `false` object in Python, not the values `True` or `False`. Let's turn to a few examples to see how this works:

```
> >> 2 < 3, 3 < 2 # Less than: return True or False (1 or 0)
(True, False)
```

Magnitude comparisons such as these return `True` or `False` as their truth results, which, as we learned in Chapters 5 and 9, are really just custom versions of the integers 1 and 0 (they print themselves differently but are otherwise the same). Boolean operators `and` and `or`, on the other hand, always return an object—either the object on the left side of the operator or the object on the right. If we test their results in `if` or other statements, they will be as expected (remember, every object is inherently `true` or `false`), but we won't get back a simple `True` or `False`. For `or` tests, Python evaluates the operand objects from left to right and returns the first one that is `true`. Moreover, Python stops at the first `true` operand it finds. This is usually called *short-circuit evaluation*, as determining a result short-ci

rcuits (terminates) the rest of the expression as soon as the result is known:

```
> >> 2 or 3, 3 or 2 # Return left operand if true

(2, 3) # Else, return right operand (true or fa...
> >> [] or 3

3
> >> [] or {}

{}
```

In the first line of the preceding example, both operands (2 and 3) are true (i.e., are nonzero), so Python always stops and returns the one on the left—which determines the result because true or anything is always true. In the other two tests, the left operand is false (an empty object), so Python simply evaluates and returns the object on the right—which may have either a true or a false value when tested, but determines the result of the or at large. Python and operations also stop (short-circuit) as soon as the result is known. However, in this case Python evaluates the operands from left to right and stops if the left operand is a false object because it determines the result—false and anything is always false:

```
> >> 2 and 3, 3 and 2 # Return left operand if false
(3, 2) # Else, return right operand (true or fa...
> >> [] and {}

[]
> >> 3 and []

[]
```

Here, both operands are true in the first line, so Python evaluates both sides and returns the object on the right—which determines the result of the `and`. In the second test, the left operand is false (`[]`), so Python stops and returns it as the `and` result without ever running the code on the right side. In the last test, the left side is true (`3`), so Python evaluates and returns the object on the right—which happens to be a false `[]`. The net effect of all this apparent nonsense is the same as in most other languages—you get a value that is logically true or false if tested in an `if` or `while` according to the normal definitions of `or` and `and`. However, in Python, Booleans return either the left or the right object, not a simple integer flag. This behavior of `and` and `or` may seem esoteric at first glance, but see this chapter's sidebar, "Why You Will Care: Booleans", for examples of how it is sometimes used to advantage in Python coding. The next section also shows a common way to leverage this behavior, and its more mnemonic alternative.

中文翻译

比较、相等和真值的概念在第 9 章已经介绍过。不过和语法一样，if 是我们学过的第一条真正用到这些工具的语句，所以我们要补充信息、重新过一遍这些概念。总而言之，Python 的布尔运算符与其他一些语言的对应物略有不同。在 Python 中：

- 所有对象都有固有的布尔真值或假值。
- 任何非零数字或非空对象都为真。
- 零数字、空对象和特殊对象 None 被视为假。
- 比较和相等测试会递归应用于数据结构。
- 比较和相等测试返回 True 或 False (1 和 0 的自定义版本)。
- 布尔 and 和 or 运算符返回一个真或假的操作数对象。
- 布尔运算符一旦知道结果就停止求值 (短路, short-circuit)。

if 语句基于真值采取行动，而布尔运算符则用于以更丰富的方式组合其他测试的结果，产生新的真值。更正式地说，Python 有三个布尔表达式运算符：

- X and Y ——当 X 和 Y 都为真时为真——并返回 X 或 Y
- X or Y ——当 X 或 Y 为真时为真——并返回 X 或 Y
- not X ——当 X 为假时为真——并返回 True 或 False

这里，X 和 Y 可以是任何真值，或任何返回真值的表达式 (例如 == 相等测试、in 成员检查等)。Python 中布尔运算符用单词拼写 (而不是 C 的 &&、|| 和 !) ，并且不要与 Python 中作用于数字、集合 (和字典) 的 &、|、^ 运算符混淆。最独特的是，Python 的布尔 and 和 or 运算符返回真或假的对象，而不是 True 或 False。让我们看几个例子来理解它的工作原理：

```
>>> 2 < 3, 3 < 2          # True False1 0
(True, False)
```

这类大小比较返回 True 或 False 作为真值结果，正如我们

在第 5、9 章所学，它们其实只是整数 1 和 0 的自定义版本（打印形式不同，其余完全相同）。而布尔运算符 `and` 和 `or` 总是返回一个对象——要么是运算符左侧的对象，要么是右侧的对象。如果在 `if` 或其他语句中测试它们的返回值，结果符合预期（记住，每个对象天生为真或假），但我们拿不到一个单纯的 `True` 或 `False`。对于 `or` 测试，Python 从左到右求值操作数对象，返回第一个为真的。而且 Python 在找到第一个真操作数时就停止。这通常被称为短路求值（short-circuit evaluation），因为一旦结果已知，求值就短路（终止）了表达式的其余部分：

```
>>> 2 or 3, 3 or 2      #
(2, 3)                  #
>>> [] or 3
3
>>> [] or {}
{}
```

在上例第一行中，两个操作数（2 和 3）都为真（即非零），所以 Python 总是停下来返回左边那个——它决定了结果，因为真 `or` 任何东西恒为真。另外两个测试中，左操作数为假（空对象），所以 Python 直接求值并返回右边的对象——它在被测试时可能是真也可能是假，但决定了整个 `or` 的结果。Python 的 `and` 操作同样在结果已知时停止（短路）。不过这种情况下，Python 从左到右求值操作数，如果左操作数是假对象就停止，因为它决定了结果——假 `and` 任何东西恒为假：

```
>>> 2 and 3, 3 and 2      #
(3, 2)                    #
>>> [] and {}
[]
>>> 3 and []
[]
```

这里，第一行两个操作数都为真，所以 Python 求值两侧并返回右边的对象——它决定了 and 的结果。第二个测试中，左操作数为假（[]），所以 Python 停止并把它作为 and 的结果返回，右侧代码完全不会执行。最后一个测试中，左侧为真（3），所以 Python 求值并返回右侧对象——碰巧是一个假的 []。所有这些看似无厘头的行为，最终效果与其他大多数语言相同——按照 or 和 and 的常规定义，你得到的是一个在 if 或 while 中测试时逻辑上为真或假的值。但在 Python 中，布尔运算符返回的是左侧或右侧的对象，而不是一个简单的整数标志。and 和 or 的这种行为乍看之下可能显得晦涩，但请参阅本章末尾的边栏“Why You Will Care: Booleans”（为什么你会关心：布尔值），看它在 Python 编程中如何被巧用。下一节还会展示利用这一行为的一种常见方式，以及它更易记的替代方案。

深度理解

·核心概念：Python 真值测试的三条铁律——1) 每个对象天生为真或假（非零、非空、非 None 即真）；2) and/or 返回对象而非布尔值；3) 一旦结果确定立即短路。

·底层实现：CPython 对 and/or 的编译方式：X and Y 编译为“求值 X → 若 X 为假跳转 → 求值 Y”；X or Y 编译为“求值 X → 若 X 为真跳转 → 求值 Y”。跳转的“目标值”就是作为结果的表达式——所以返回的永远是操作数对象

本身。真值判定最终调用对象的 bool（或退而求其次 len）方法，C 层面的 PyObjectIsTrue() 函数完成这件事。

·设计原因：返回对象 + 短路，源自 Lisp 的传统（(or a b) 返回第一个非空值）。这让布尔表达式可以“顺路取值”，一个表达式同时完成“判断”和“取第一个可用值”两件事。

·实际问题：X = A or B or C or None（取第一个非空值）是 Python 最常用的惯用法之一，见章末边栏。

·初学者误区：1) 以为 2 and 3 返回 True——它返回 3；2) 把 &/|（位运算）与 and/or（逻辑运算）混淆；3) 写 if x is True: 或 if x == True: 判断“真值”——直接 if x: 才是正道；4) 忘记短路会导致右侧函数不执行（副作用丢失）。

代码分析

```
>>> 2 < 3, 3 < 2          # True False1 0
(True, False)
>>> 2 or 3, 3 or 2        #
(2, 3)                    #
>>> [] or 3               # [] -> 3
3
>>> 2 and 3, 3 and 2      #
(3, 2)                    #
>>> [] and {}             # [] -> []
[]
>>> 3 and []              # 3 -> []
[]
```

解释器如何处理：2 or 3 编译后的字节码逻辑是：压入 2，执行 POPJUMPIFTRUE（跳转到取 2 作为结果的指令

)，否则压入 3。因为 2 非零为真，跳转直接发生，3 根本不会被求值。[] or 3 中 [] 为假（长度 0），跳转不发生，压入 3 作为结果。and 恰好相反：POPJUMPIFFALSE。这些指令的执行都基于 PyObjectIsTrue() 的 C 调用，它依次尝试对象的 bool 和 len 来确定真伪——这就是为什么 []、{}、''、0、None 全为假，它们要么 bool 返回 False，要么 len 返回 0。

设计思想："返回对象"让布尔运算从"判断题"升级为"选择题"。理解 or 的本质是"返回第一个真值对象"而非"返回 True"，是打通 Python 惯用法（默认值、链式取值）的钥匙。

12.7 if/else 三元表达式 (The if/else Ternary Expression)

英文原文

One common role for the prior section's Boolean operators is to code an expression that runs the same as an if statement. To get started, consider the following very common code, which sets A to either Y or Z, based on the truth value of X:

```
if X:
    A = Y
else:
    A = Z
```

Sometimes, though, the items involved in such a statement are so simple that it seems like overkill to spread them across four lines. At other times, we may want to nest such a construct in a larger statement instead of assigning its result to a variable separately. For such reasons (and possibly to appease ex-C programmers), Python includes a "ternary" (three-part) expression that allows us to say the same thing in just one line of code:

```
A = Y if X else Z
```

This expression has the exact same effect as the preceding four-line if statement, but it's simpler to code. In some sense, it is to if statements what the prior chapter's `:=` is to assignment statements: an expression equivalent with more limited syntax and narrower roles, which is nevertheless useful in some code. For example, you can't code full statements in the parts of this expression, but you can embed it anywhere that Python allows an expression. As in the statement equivalent, the ternary expression runs expression Y only if X turns out to be true and runs expression Z only if X turns out to be false. That is, it short-circuits, just like the Boolean operators described in the prior section, running just Y or Z but not both. Here are some examples of it in action:


```
> >> tone = 'formal'>> a = 'code' if tone == 'formal' else 'hac...
'code'
> >> tone = 'informal'>> a = 'code' if tone == 'formal' else 'hack...
'hack'
```

The same effect can often be achieved by a careful combination of `and` and `or` operators, because they return either the object on the left side or the object on the right as the preceding section described. The ternary expression in the following works the same as the Boolean expression below it:

```
A = Y if X else Z           # Ternary if/else
A = ((X and Y) or Z)       # and+or equivalent
```

This works, but there is a catch—you have to be able to assume that `Y` will be Boolean `true`. If that is the case, the effect is the same: the `and` runs first and returns `Y` if `X` is true; if `X` is false the `and` skips `Y` and returns `false`, and the `or` simply returns `Z`. In other words, we get “if `X` then `Y` else `Z`”—which is exactly what the ternary expression says, albeit in a different order. The `and/or` combination form also seems to require a “moment of great clarity” to understand the first time you see it, which qualifies as an argument against its deployment. As a guideline: use the equivalent and more robust and mnemonic `if/else` expression when you need this structure, or use a full `if` statement when the parts are nontrivial. As a side note (and just in case this section hasn't made your head explode yet),

using the following expression is similar to the prior Boolean and ternary expressions, because the bool function will translate any X into the equivalent of integer 1 or 0 (i.e., True or False), which can then be used as an offset to pick true and false values from a list:

```
A = [Z, Y][bool(X)]
```

Truth be told, the bool is not required when X already yields a truth value, but is when X is an object like a string:

```
> >> ['hack', 'code'][tone == 'formal']
```

```
'hack'
```

```
> >> ['hack', 'code'][bool('formal')]
```

```
'code'
```

But alas, this isn't exactly the same, because Python will not short-circuit—it will always run both Z and Y, regardless of the value of X. Because of such complexities, you're better off using the simpler and more easily digested if/else expression. Even then, common sense goes a long way here as always. Like most nestable tools, the ternary expression is naturally prone to yield code that's tough to read. If you find yourself working hard at packing logic into one, consider taking a moment to think about how hard it will be to unpack later. Your coworkers will be glad you did.

中文翻译

上一节布尔运算符的一个常见用途，是编写一条与 if 语句效果相同的表达式。先看下面这段非常常见的代码——它根据 X 的真值，把 A 设为 Y 或 Z：

```
if X:
    A = Y
else:
    A = Z
```

但有时候，这条语句涉及的元素太简单，分成四行显得有些小题大做。另一些时候，我们想把这种结构嵌套进更大的语句里，而不是把结果单独赋给变量。出于这些原因（也许还有安抚前 C 程序员的因素），Python 提供了一个“三元”（三部分）表达式，让我们只用一行代码就能表达同样的事：

```
A = Y if X else Z
```

这个表达式与前面四行的 if 语句效果完全相同，但写起来更简单。从某种意义上说，它之于 if 语句，就像上一章的：
= 之于赋值语句：一种表达式等价物，语法更受限、角色更狭窄，但在某些代码中依然有用。例如，你不能在这个表达式的各部分里写完整语句，但你可以把它嵌入到 Python 允许出现表达式的任何地方。与等价的语句一样，三元表达式只在 X 为真时运行表达式 Y，只在 X 为假时运行表达式 Z。也就是说，它会短路——和上一节描述的布尔运算符一样，只运行 Y 或 Z 中的一个，而不是两者都运行。下面是一些实例：

```
>>> tone = 'formal'
>>> a = 'code' if tone == 'formal' else 'hack'
>>> a
'code'
>>> tone = 'informal'
>>> a = 'code' if tone == 'formal' else 'hack'
>>> a
'hack'
```

同样的效果常常可以用 `and` 与 `or` 运算符的巧妙组合来实现，因为如上一节所述，它们返回的是左侧或右侧的对象。下面的三元表达式与它下方的布尔表达式效果相同：

```
A = Y if X else Z          # if/else
A = ((X and Y) or Z)      # and+or
```

这能行，但有个前提——你必须能假定 `Y` 为布尔真。如果满足这一点，效果就相同：`and` 先运行，若 `X` 为真就返回 `Y`；若 `X` 为假，`and` 跳过 `Y` 并返回假的 `X`，随后 `or` 直接返回 `Z`。换句话说，我们得到“if `X` then `Y` else `Z`”——正是三元表达式所说的，只是顺序不同。`and/or` 组合形式第一次见到时似乎也需要“灵光一闪”才能看懂，这本身就构成了反对使用它的论据。作为准则：需要这种结构时，使用等价、更稳健、更易记的 `if/else` 表达式；当各部分较复杂时，使用完整的 `if` 语句。作为题外话（万一本节还没让你的脑袋爆炸）：下面的表达式与前面的布尔表达式和三元表达式类似，因为 `bool` 函数会把任何 `X` 转换成整数 1 或 0 的等价物（即 `True` 或 `False`），然后可以把它当作偏移量，从列表中挑选真值和假值：

```
A = [Z, Y][bool(X)]
```

说实话，当 `X` 本身已经产生真值时，`bool` 不是必需的；但当 `X` 是字符串这类对象时就需要：

```
>>> ['hack', 'code'][tone == 'formal']
'hack'
>>> ['hack', 'code'][bool('formal')]
'code'
```

但唉，这并不完全一样，因为 Python 不会短路——无论 X 的值如何，它总会同时运行 Z 和 Y。由于这些复杂性，你最好使用更简单、更容易消化的 if/else 表达式。即使如此，一如既往，常识能帮上大忙。与大多数可嵌套工具一样，三元表达式天生容易产生难以阅读的代码。如果你发现自己正费尽心机把逻辑塞进一个三元表达式，不妨停下来想一想：以后要把它们拆开时会有多难。你的同事会感激你的。

深度理解

·核心概念：Y if X else Z 是一行版 if/else：先测 X，真则求值 Y，假则求值 Z，只运行其一（短路）。注意顺序是“结果 if 条件 else 结果”——与英语自然语序一致，与 C 的 X ? Y : Z 顺序相反。

·底层实现：三元表达式编译为与 if/else 相同的“跳转”字节码：求值 X → 真跳转取 Y 分支、假取 Z 分支，最终都汇合到同一出口（结果压栈）。由于表达式求值后只有一种结果进栈，它可以被任意嵌入更大的表达式。

·设计原因：与 := 的定位相同——给“简单二选一赋值”提供一个表达式形态，让代码紧凑且可嵌套；但刻意不做成完整语句的替代品。

·实际问题：默认值选择、格式化输出 (f"{x} is {'even' if x%2==0 else 'odd'}")、列表推导式内的条件映射——三元表达式在这类“内联小选择”里几乎不可替代。

·初学者误区：1) 记错顺序写成 $X ? Y : Z$ 的 C 风格——Python 顺序是 $Y \text{ if } X \text{ else } Z$ ；2) 用 $(X \text{ and } Y) \text{ or } Z$ 模仿三元——当 Y 为假值时结果错误；3) 嵌套多层三元 ($a \text{ if } b \text{ else } c \text{ if } d \text{ else } e$) ——可读性灾难，作者明确劝阻。

代码分析

```
tone = 'formal'
a = 'code' if tone == 'formal' else 'hack'    # -> a = 'code'
tone = 'informal'
a = 'code' if tone == 'formal' else 'hack'    # -> a = 'hack'

A = Y if X else Z                            # if/else
A = ((X and Y) or Z)                         # and+or Y
A = [Z, Y][bool(X)]                          #
```

解释器如何处理： $a = 'code' \text{ if } tone == 'formal' \text{ else } 'hack'$ 编译后：求值 $tone == 'formal'$ （比较运算，返回 True/False），POPJUMPIFFALSE 决定压入 'code' 还是 'hack'，最后 STORENAME a 把栈顶值存进变量。整个选择发生在求值阶段，只压入被选中的那个常量。而 $[Z, Y][bool(X)]$ 编译为先构造 $[Z, Y]$ 列表（两个元素都被求值！若 Z 或 Y 是函数调用，副作用必然发生），再做下标索引——这就是它“不短路”的根源。

设计思想：三种写法表达同一逻辑，但正确性、可读性、性能各不相同。作者的结论掷地有声：用最简单的那个—— $Y \text{ if } X \text{ else } Z$ 。这也是 Python 之禅“简单优于复杂”在表达式层面的缩影。

12.8 本章小结 (Chapter Summary)

英文原文

In this chapter, we studied the Python if statement. Additionally, because this was our first compound and logical statement, we reviewed Python's general syntax rules and explored the operation of truth values and tests in more depth than we were able to previously. Along the way, we also looked at how to code multiple-choice logic in Python with both dictionaries and match, learned about the if/else ternary expression, and explored Boolean operators. The next chapter continues our look at procedural statements by expanding the coverage of while and for loops. There, you'll learn about alternative ways to code loops in Python, some of which may be better than others. Before that, though, here is the usual chapter quiz to review before moving ahead.

中文翻译

在本章中，我们学习了 Python 的 if 语句。此外，因为这是我们的第一条复合语句和逻辑语句，我们还复习了 Python 的通用语法规则，并比之前更深入地探索了真值和测试的运作方式。一路上，我们还看了如何用字典和 match 在 Python 中编写多选逻辑，学习了 if/else 三元表达式，并探索了布尔运算符。下一章继续我们对过程式语句的考察，扩展 while 和 for 循环的覆盖。在那里，你会学到在 Python

中编写循环的多种方式，其中有些可能比其他的更好。不过在那之前，照例先做一下本章测验，复习后再继续前进。

深度理解

·核心概念：本章四件大事——if 语句、语法模型（缩进与续行）、真值/布尔运算、三元表达式与多选（字典/match）。

·结构安排：作者特意把"语法模型"插在 if 和 match 之后、真值之前讲，是因为 if 是第一个真正用到这些机制的语句——先遇到问题，再讲解决方案。

·衔接：下一章 while/for 循环会继续使用真值测试和复合语句语法，因此本章是"循环"的语法与逻辑地基。

12.9 测验 (Test Your Knowledge: Quiz)

英文原文

1. How might you code a multiple-choice branch in Python? 2. How can you code an if/else statement as an expression in Python? 3. How can you make a single statement span many lines? 4. What do the words True and False mean? 5. What does "short-circuiting" mean, and where does it crop up?

中文翻译

1. 在 Python 中，你会怎样编写一个多选分支？2. 在 Python 中，如何把 if/else 语句写成表达式？3. 如何让一条语

句跨越多行？4. 单词 True 和 False 是什么意思？5. "短路" (short-circuiting) 是什么意思？它出现在哪些地方？

12.10 测验答案 (Test Your Knowledge: Answers)

英文原文

1. An if statement with multiple elif clauses is often the most straightforward way to code a multiple-choice branch, though not necessarily the most concise or flexible. Dictionary indexing can often achieve the same result, especially if the dictionary contains callable functions coded with def statements or lambda expressions. As of Python 3.10, the match statement provides explicit syntax for multiple-choice selections; it works well in basic roles but can't code logic as general as that in the if statement and comes with substantial heft in support of structural pattern matching, a very different tool. 2. The expression form Y if X else Z returns Y if X is true, or Z otherwise; it's the same as a four-line if statement and works well in limited contexts, but can't code actions as rich as those in the full if statement, and has the potential to produce code that's hard to read. The and/or combination (((X and Y) or Z)) can work the same way, but it's more obscure and requires that the Y part be true. 3. Wrap up the statement in an open syntactic pair ((), [], or {}), an

d it can span as many lines as you like; the statement ends when Python sees the closing (right) half of the pair, and lines 2 and beyond of the statement can begin at any indentation level. Backslash continuations work, too, but are broadly discouraged in the Python world. 4. This is partly a review from Chapter 9, but is reinforced in the sidebar at the end of this chapter. True and False are just custom versions of the integers 1 and 0, respectively: they always stand for Boolean true and false values in Python. They're available for use in truth tests and variable initialization and are printed for expression results at the interactive prompt. In all these roles, they serve as a more mnemonic and hence readable alternative to 1 and 0. 5. Short-circuiting happens when Python stops evaluating an expression early because its result can already be determined from the expression so far. It comes up in and and or expressions, which run their right side only if their left sides don't determine their results. It also comes up in the if/else ternary expression, which runs either its true or false parts, depending on its test part's logical result.

中文翻译

1. 带多个 elif 子句的 if 语句通常是编写多选分支最直白的方式，虽然不一定最简洁或最灵活。字典索引常常能达到同样的效果，尤其是当字典里存放着用 def 语句或 lambda 表达式编写的可调用函数时。从 Python 3.10 起，match

语句为多选提供了显式语法；它在基础角色上表现良好，但不能像 if 语句那样编写任意通用的逻辑，而且为了支持结构模式匹配（一个截然不同的工具）背负了相当可观的分量。

2. 表达式形式 `Y if X else Z` 在 `X` 为真时返回 `Y`，否则返回 `Z`；它与四行的 if 语句等价，在有限的场景下表现良好，但不能表达完整 if 语句那样丰富的动作，而且有产出难读代码的潜力。`and/or` 组合 (`((X and Y) or Z)`) 也能达到同样的效果，但它更晦涩，而且要求 `Y` 部分为真。

3. 把语句包进一个打开的语法括号对 (`()`、`[]` 或 `{}`)，它就可以跨任意多行；当 Python 看到括号对的右半部分 (`()`、`}` 或 `]`) 时语句才结束，而且语句的第 2 行及以后可以从任何缩进层级开始。反斜杠续行也行，但在 Python 世界中普遍不被推荐。

4. 这有一部分是对第 9 章的复习，但本章末尾的边栏会加强它。`True` 和 `False` 分别只是整数 1 和 0 的自定义版本：它们在 Python 中永远代表布尔真和假。它们可用于真值测试和变量初始化，也会在交互提示符下作为表达式结果显示。在所有这些角色中，它们都是比 1 和 0 更易记、因而更可读的替代品。

5. 当 Python 因为“结果已经能从现有表达式部分确定”而过早停止求值时，就发生了短路。它出现在 `and` 和 `or` 表达式中——只有当左侧没有决定结果时，才会运行右侧。它也出现在 if/else 三元表达式中——根据测试部分的逻辑结果，只运行真或假分支之一。

12.11 边栏：为什么你会关心：布尔值 (Why You Will Care: Booleans)

英文原文

One common way to use the somewhat unusual behavior of Python Boolean operators is to select from a set of objects with an `or`. A statement such as this:

```
X = A or B or C or None
```

assigns `X` to the first nonempty (that is, `true`) object among `A`, `B`, and `C`, or to `None` if all of them are empty. This works because the `or` operator returns one of its two objects, and it turns out to be a fairly common coding paradigm in Python: to select a nonempty object from among a fixed-size set, simply string them together in an `or` expression. In simpler form, this is also commonly used to designate a default—the following sets `X` to `A` if `A` is `true` (nonzero or nonempty), and to default otherwise:

```
X = A or default
```

It's also important to understand the short-circuit evaluation of Boolean operators and the `if/else`, because it may prevent actions from running. Expressions on the right of a Boolean operator, for example, might call functions that perform substantial or important work, or have side effects that won't happen if the short-circuit rule takes effect:

```
if f1() or f2(): ...
```

Here, if `f1` returns a `true` (or nonempty) value, Python will never run `f2`. To guarantee that both functions wi

ll be run, call them before the or:

```
tmp1, tmp2 = f1(), f2()
if tmp1 or tmp2: ...
```

You've already seen another application of this behavior in this chapter: because of the way Booleans work, the expression `((A and B) or C)` can be used to emulate an if statement—almost (see this chapter's discussion of this form for details). We met additional Boolean use cases in prior chapters. As we saw in Chapter 9, because all objects are inherently true or false, it's common and easier in Python to test an object directly (`if X:`) than to compare it to an empty value (`if X!="":`). For a string, the two tests are equivalent. As we also saw in Chapter 5, the preset Boolean values `True` and `False` are the same as the integers 1 and 0 and are useful for initializing variables (`X = False`), for loop tests (`while True:`), and for displaying results at the interactive prompt. Also watch for related discussion in operator overloading in Part VI: when we define new object types with classes, we can specify their Boolean nature with either the `bool` or `len` methods. The latter of these is tried if the former is absent and designates false by returning a length of zero—because an empty object is considered false. Finally, and as a preview, other tools in Python have roles similar to the `or` chains at the start of this sidebar: the `filter` call and list comprehensions you'll explore later can be used to select true values when the set of candidates isn't known.

wn until runtime (though they evaluate all values and return all that are true), and the any and all built-ins can be used to test if any or all items in a collection are true (they short-circuit their checks like and, or, and if/else, but don't select an item per se):

```
> >> L = [1, 0, 2, 0, 'hack', '', 'py', []] >> list(filter(bool...  
  
[1, 2, 'hack', 'py']  
> >> [x for x in L if x] # Comprehensions  
  
[1, 2, 'hack', 'py']  
> >> any(L), all(L) # Aggregate truth  
  
(True, False)
```

As we've learned, the bool function here simply returns its argument's true or false value, as though it were tested in an if. Watch for more on these related tools in Chapters 14, 19, and 20.

中文翻译

利用 Python 布尔运算符有点不寻常的行为的一种常见方式，是用 or 从一组对象中做选择。像这样的一条语句：

```
X = A or B or C or None
```

会把 X 赋为 A、B、C 中第一个非空（即为真）的对象；如果它们全都为空，则赋为 None。这能成立是因为 or 运算符返回它的两个操作数之一，而这在 Python 中恰好是一种相当常见的编码范式：想从一组固定规模的对象中选出一个非空对象，只需把它们用 or 串起来。以更简单的形式，这也常被用来指定默认值——下面把 X 设为 A（若 A 为真，

即非零或非空)，否则设为 default：

```
X = A or default
```

理解布尔运算符和 if/else 的短路求值也很重要，因为它可能阻止某些动作运行。例如，布尔运算符右侧的表达式可能调用执行大量或重要工作的函数，或带有在短路规则生效时不会发生的副作用：

```
if f1() or f2(): ...
```

这里，如果 f1 返回真（或非空）值，Python 永远不会运行 f2。要保证两个函数都被运行，就在 or 之前调用它们：

```
tmp1, tmp2 = f1(), f2()
if tmp1 or tmp2: ...
```

你在本章已经看到这一行为的另一个应用：因为布尔运算的工作方式，表达式 ((A and B) or C) 可以用来模拟 if 语句——几乎（详见本章对这种形式的讨论）。我们在前面的章节还遇到过其他布尔用例。如第 9 章所见，由于所有对象天生为真或假，在 Python 中直接测试对象 (if X:) 比与空值比较 (if X != "") 更常见、更容易。对于字符串，这两种测试是等价的。如第 5 章所见，预设的布尔值 True 和 False 与整数 1 和 0 相同，可用于初始化变量 (X = False)、循环测试 (while True:) 以及交互提示符下的结果显示。还要留意第六部分中运算符重载的相关讨论：当我们用类 (class) 定义新对象类型时，可以用 bool 或 len 方法指定它们的布尔特性。前者缺席时会尝试后者，而后者通过返回长度为 0 来标记为假——因为空对象被视为假。最后，作为一个预告，Python 中还有其他工具扮演着与本边栏开头 or 链类似的角色：你以后会学到的 filter 调用和列表推导式 (list comprehension) 可以在候选集合直到运行时

才已知时用来选出真值（虽然它们会求值所有值并返回所有为真的）；而 `any` 和 `all` 内建函数可以用来测试集合中是否有任意项或所有项为真（它们像 `and`、`or` 和 `if/else` 一样短路检查，但本身并不挑选某个项）：

```
>>> L = [1, 0, 2, 0, 'hack', '', 'py', []]
>>> list(filter(bool, L))           #
[1, 2, 'hack', 'py']
>>> [x for x in L if x]             #
[1, 2, 'hack', 'py']
>>> any(L), all(L)                 #
(True, False)
```

如我们所学，这里的 `bool` 函数只是返回其参数的真或假值，就像在 `if` 中被测试一样。这些相关工具的更多内容，请关注第 14、19、20 章。

深度理解

- 核心概念：布尔运算符最有价值的三个实战用法——`A or B or C or None` 链式取第一个非空值、`X = A or default` 给默认值、警惕短路吞掉副作用。
- 底层实现：`X = A or B or C or None` 在字节码层面是一串条件跳转：`A` 真则跳转到赋值，否则求 `B`.....链越长，跳转越多，但都只求值到“第一个非空对象”为止。`any()/all()` 内建是 C 语言实现的聚合短路版本。
- 设计原因：`or` 返回对象的设计让“取默认值”不必写三行 `if`——这正呼应了“Python 代码量是 Java/C++ 的 1/3”的卖点。
- 实际问题：配置读取（`port = config.get('port') or 8080`）、函数参数兜底（`name = name or 'guest'`）、多层

回退查找——全是 or 链的天下。

·初学者误区：1) 把 `X = A or default` 当魔法——它不过是短路求值的自然结果；2) 在 `if f1() or f2():` 里假设两个函数都会执行——`f1` 为真时 `f2` 静默缺席；3) 对"可能是合法空值"的数据用 `or` 兜底——如 `x = 0 or 100` 得到 `100`，`0` 被误伤；此时应该用 `if x is None:` 这类显式判断。

代码分析

```
X = A or B or C or None      # ABC None
X = A or default              # A A default

if f1() or f2(): ...         # f1 f2
tmp1, tmp2 = f1(), f2()      #
if tmp1 or tmp2: ...

L = [1, 0, 2, 0, 'hack', '', 'py', []]
list(filter(bool, L))        # filter [1, 2, 'hack', 'py']
[x for x in L if x]          #
any(L), all(L)               # -> (True, False)
```

解释器如何处理：`X = A or B or C or None` 编译为一系列 `POPJUMPIFTRUE` 跳转：求值 `A`，为真则跳过后续所有求值直接存给 `X`；为假则求值 `B.....None` 是终点兜底。
`if f1() or f2():` 中，`f1` 返回真时，`f2` 的调用指令被跳转绕过——`f2` 的函数体根本没有执行，这是字节码层面的"整段跳过"，不是"执行了但忽略结果"。
`any()` 内部同样短路：遇到第一个真值立即返回 `True`，绝不扫描整个集合；`all()` 相反，遇到第一个假值返回 `False`。
`bool` 函数则直接调用对象的 `bool/len` 得到 `0` 或 `1`。

设计思想：短路不是 Python 的怪癖，而是几乎所有语言

的统一约定（C、Java、JS 的 `&&/||` 都短路）。把它理解为“语言保证的最小求值”，才能既享受它的便利，又避免它吞掉副作用的坑。

本章总结

技术拓展 (Technical Expansion)

1. 实际项目中的应用场景

- 配置与参数兜底：`timeout = env.get('TIMEOUT') or 30`——`or` 链是配置文件解析的第一惯用法（但注意 0 值会被误伤，此时用显式 `is None` 判断）。
- 路由与命令分发：Web 框架（FastAPI/Django）的路由表和 CLI 工具（`argparse` 之外的手写分发）都是“字典跳转表”或 `match` 的用武之地。
- 状态机与解析器：协议报文解析、JSON 结构提取、AST 节点处理——`match` 的序列/映射/实例模式在这里真正发光（如 `case {"type": "text", "content": text}`）。
- 表单校验与业务分支：`if/elif` 链处理表单字段、权限级别、支付状态等“互斥分支”，是每天出现最多的 `if` 场景。
- 数据清洗：`list(filter(bool, data))`、`[x for x in data if x]` 一行剔除空值，比手写循环简洁得多。

2. 与其他语言 (Java/C++) 的区别

维度	Pytho	Java	C++				
块定界	缩进 (语法级)	{ }	{ }				
布尔运算	and/or/not (单词)	&&/\	\	/!	&&/\	\	/!
布尔返回值	返回操作数对	只返回 boole	只返回 bool				
三元表达式	Y if X else Z	X ? Y : Z	X ? Y : Z				
switch	无 (用 match , 3.10 +)	switch (旧) → switch 表达式/模式匹配 (	switch/case, 无模式匹配				
短路	有 (与 C 家族相同)	有	有				
多选默认	dict.get(key, defa	switch default:	switch default:				
类型检查	运行时 (鸭子类型)	编译期 (静态)	编译期 (静态)				

核心洞察：Python 与 C 家族在"选择语句"上的最大分歧有两点：其一，Python 的布尔运算返回对象而非标志位——2 or 3 是 2 而不是 true；其二，C/Java 的 switch 只按"值相等"选择，Python 的 match 是"结构模式匹配"——不仅能比值，还能拆结构、绑变量。而 Java 17+ 和 C++17 也开始向"模式匹配"演进，说明这是语言发展的共同方向。

3. Python 发展历史背景

- 1991–2021（三十年无 switch）：Python 一直用 if/elif 和字典解决多选，官方立场是“已有工具足够”。
- 2020 年 PEP 622：Thomas Wouters 等人提出结构模式匹配草案，社区辩论激烈——支持者要表达力，反对者（包括本书作者 Mark Lutz）嫌复杂度。
- 2021 年 Python 3.10 正式发布：PEP 634（语法）、PEP 635（原理）、PEP 636（教程）落地，match/case 成为关键字（3.9 中还是“软关键字”）。
- 后续演进：3.11+ 对模式匹配有性能优化；3.12 保持语法稳定。match 是 Python 3.x 时代最后的大型语法新增之一。

4. 高级开发者需要掌握的相关知识

- 字节码视角：用 dis 模块反汇编 if、match、三元表达式，观察 POPJUMPIFFALSE、MATCH 系列指令的差异——这是理解“短路”和“模式匹配成本”的捷径。
- bool 与 len：自定义类的真值语义（bool 优先，缺失时退回 len）；写 ORM 模型或容器类时这是必考点。
- match 的完整语法：类模式（case Point(x=0, y=0)）、位置参数捕获、守卫的“失败不算命中”语义、case 与裸变量的区别——按需查阅 PEP 634。
- 代码质量：if x is True / if x == None 是反模式（PEP 8 建议 if x is None）；三元表达式嵌套不超过一层；(X and Y) or Z 的历史写法在现代代码中应当避免。

·类型标注配合：typing.Literal 与 match 的组合、Never 类型用于穷尽性检查（exhaustiveness check）——类型检查器（mypy）能验证 match 是否覆盖了所有分支。

学习建议 (Learning Advice)

·重要程度：★★★★☆ (4/5) 。if 是真值测试与后续所有控制流的基石；match 则了解为主——它是 3.10+ 的重要语法，但日常代码里 if/字典仍占主流。

·应该掌握到什么程度：

·必会：if/elif/else 的完整语法与执行规则；缩进/续行/分号三条语法规则；真值测试（非零、非空、None 为假）；and/or 短路且返回对象；三元表达式 Y if X else Z；dict.get() 做多选默认。

·掌握：match 基础用法（字面量、`|`、`as`、裸变量捕获）；字典跳转表（值存函数再调用）。

·了解：match 高级模式（序列/映射/属性/实例/守卫）——知道存在、会用兜底即可，需要时查手册。

·后续学习内容：

·立即：第 13 章 while/for 循环——会继续使用真值测试和复合语句语法，是本章知识的直接延续。

·近程：第 14 章迭代器与推导式（filter、any/all 的正式舞台）；第 19 章函数（跳转表的 lambda/def 完整形态）。

·进阶：第六部分运算符重载的 bool/len；CPython 源码

中 `ceval.c` 的跳转指令实现；PEP 634-636 原文。

·实践建议：1) 在 REPL 里敲一遍本章全部示例，重点观察 `2 or 3`、`[]` and `{}` 这类"返回值"；2) 用 `dis.dis` 反汇编一个 `if` 和一个 `match`，亲眼看看跳转指令；3) 把 `matchdemo.py` 存成文件运行，逐个改变 `state` 的取值体会"首真即停"；4) 用 `python -m tabnanny` 检查自己写过代码的 `tab/空格混用` 问题。

本章完。下一章：while and for Loops (while 与 for 循环)